



CS240 Algorithm Design and Analysis

Lecture 19

Amortized Analysis (Cont.)

Qingying Hao
Spring 2026
2026.05.12



Time: May 20 Wednesday 10:00 – 11:00 AM

Location: 信息学院 IC 502

Title:

Misleading Large Language Models used (or misused) in Scientific Peer-Reviewing via Hidden Prompt-Injection Attacks



Giovanni Apruzzese is an Assistant Professor at the University of Reykjavik (Iceland) and the University of Liechtenstein. His research focuses on cybersecurity and trustworthy AI, with particular interests in adversarial machine learning, phishing and intrusion detection, network security, and the security implications of large language models (LLMs). His recent work also explores human-centered security and the misuse of AI systems in real-world applications.

He is the General Chair of IEEE SaTML '27 and an Associate Editor for ACM T-AI-SAP. He serves as the Vice PC Co-chair for USENIX Security '26 and PC Co-chair for AISec '26. Additionally, he is an Area Chair for NeurIPS '26 and a PC member for IEEE S&P, USENIX Security, AsiaCCS, and ACSAC.





Three Methods of Amortized Analysis



Aggregate analysis:

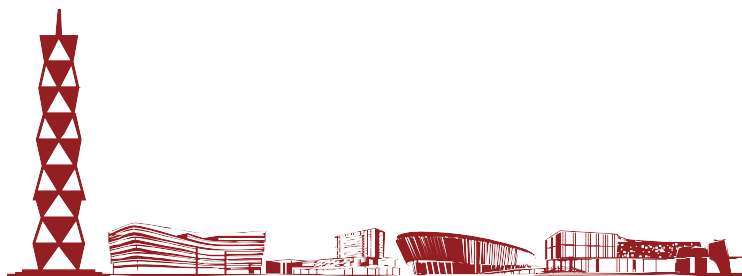
- Total cost of n operations / n

Accounting method:

- Assign each type of operation a (possibly different) amortized cost
- Overcharge some operations
- Store the overcharge as credit on specific objects
- Then use the credit for compensation for some later operations

Potential method:

- Same as accounting method
- But store the credit as “potential energy” as a whole





Fibonacci Heaps

A classical example where different types of operations have different amortized costs





Fibonacci Heaps

- A Fibonacci heap is a type of heap that implements certain operations faster than a binary heap, in amortized time.
 - The time complexities of the circled operations are amortized. The rest are worst case.
- It can be used to speed up a number of graph algorithms asymptotically.
- Both Dijkstra's and Prim's algorithms take $O((V+E) \log V)$ time with a binary heap, and $O(E + V \log V)$ time with a Fibonacci heap.
 - Both algorithms decrease the key value heap items $O(E)$ times.
 - This takes $O(E \log V)$ time on a binary heap, and $O(E)$ time on a Fibonacci heap.
- Fibonacci heaps are more complicated than binary heaps, and often don't perform better in practice.

Procedure	Binary heap (worst-case)	Fibonacci heap (amortized)
MAKE-HEAP	$\Theta(1)$	$\Theta(1)$
INSERT	$\Theta(\lg n)$	$\Theta(1)$
MINIMUM	$\Theta(1)$	$\Theta(1)$
EXTRACT-MIN	$\Theta(\lg n)$	$O(\lg n)$
UNION	$\Theta(n)$	$\Theta(1)$
DECREASE-KEY	$\Theta(\lg n)$	$\Theta(1)$
DELETE	$\Theta(\lg n)$	$O(\lg n)$

DIJKSTRA(G, w, s)

```
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  $S = \emptyset$ 
3  $Q = G.V$ 
4 while  $Q \neq \emptyset$ 
5    $u = \text{EXTRACT-MIN}(Q)$ 
6    $S = S \cup \{u\}$ 
7   for each vertex  $v \in G.Adj[u]$ 
8     RELAX( $u, v, w$ )
RELAX( $u, v, w$ )
1 if  $v.d > u.d + w(u, v)$ 
2    $v.d = u.d + w(u, v)$ 
3    $v.\pi = u$ 
```

MST-PRIM(G, w, r)

```
1 for each  $u \in G.V$ 
2    $u.key = \infty$ 
3    $u.\pi = \text{NIL}$ 
4  $r.key = 0$ 
5  $Q = G.V$ 
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G.Adj[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.key$ 
10       $v.\pi = u$ 
11       $v.key = w(u, v)$ 
```



Fibonacci Heaps: Structure

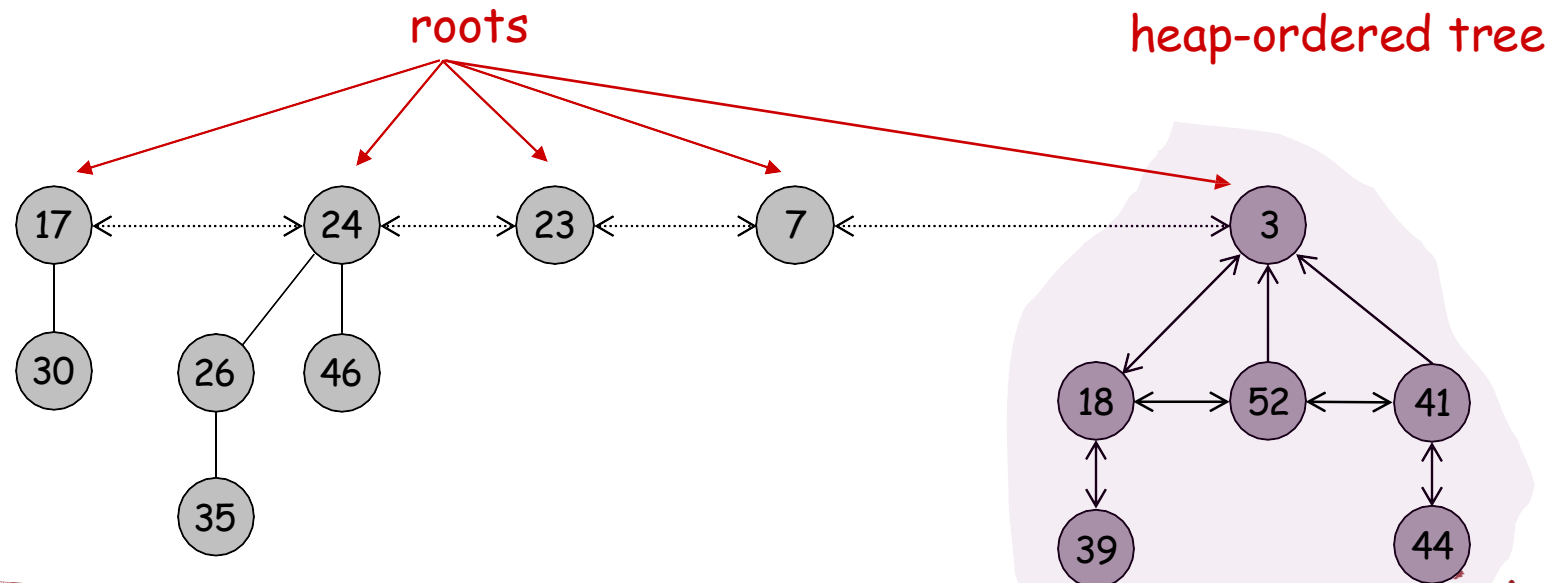


each parent smaller than its children

Fibonacci heap.

- A forest of **heap-ordered** trees; Each tree satisfies the min heap property, i.e., each node's key is less than those of all its children; store the roots in a list.
- Maintain pointer to minimum element.
- Set of marked nodes.

Each node stores an up pointer to its parent, a "down" pointer to one of its children.
Siblings are stored in a doubly linked list





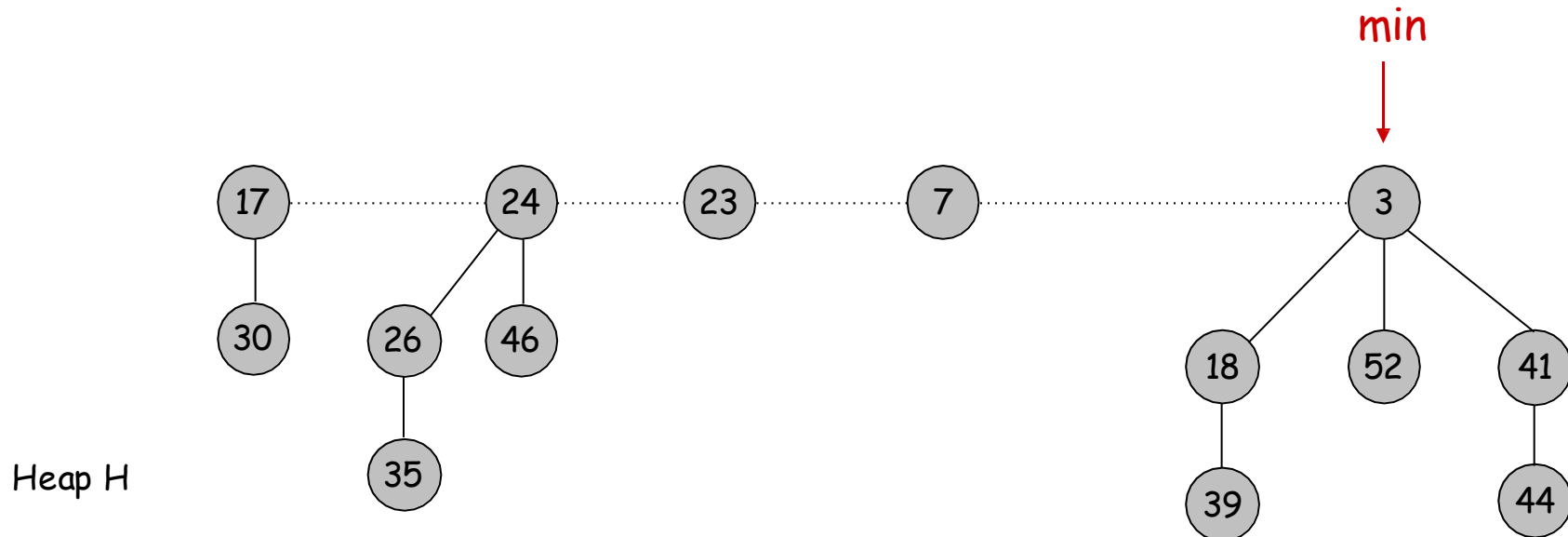
Fibonacci Heaps: Structure



Fibonacci heap.

- A forest of heap-ordered trees; Each tree satisfies the min heap property, i.e., each node's key is less than those of all its children; store the roots in a list.
- Maintain pointer to minimum element.
- Set of marked nodes.

find-min takes $O(1)$ time





Fibonacci Heaps: Structure

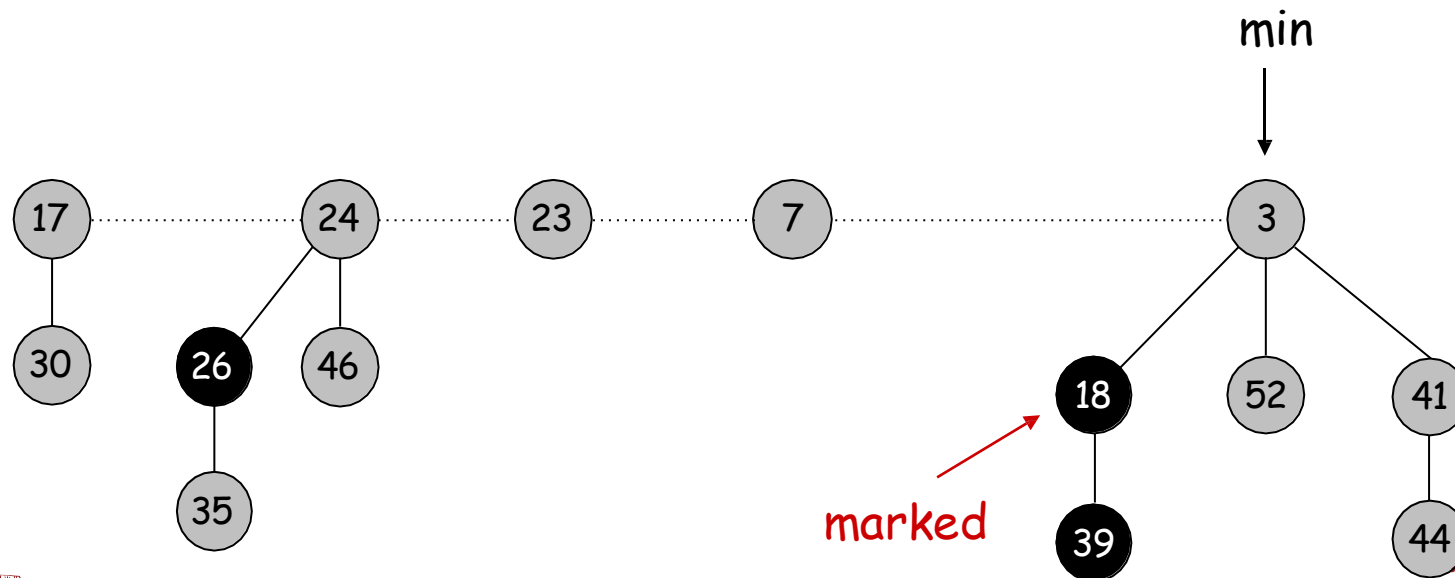


Fibonacci heap.

- A forest of heap-ordered trees; Each tree satisfies the min heap property, i.e., each node's key is less than those of all its children; store the roots in a list.
- Maintain pointer to minimum element.
- Set of marked nodes.

used to bound maximum degree (will explain later);
roots are never marked

Heap H





Fibonacci Heaps: Notation



Notation.

- n = number of nodes in heap.
- $d(x)$ = number of children of node x (degree). $D(n)$ = max degree of any node in a heap of size n . Will guarantee $D(n) = O(\log n)$
- $\text{trees}(H)$ = number of trees in heap H .
- $\text{marks}(H)$ = number of marked nodes in heap H .
-

$\text{trees}(H) = 5$

$\text{marks}(H) = 3$

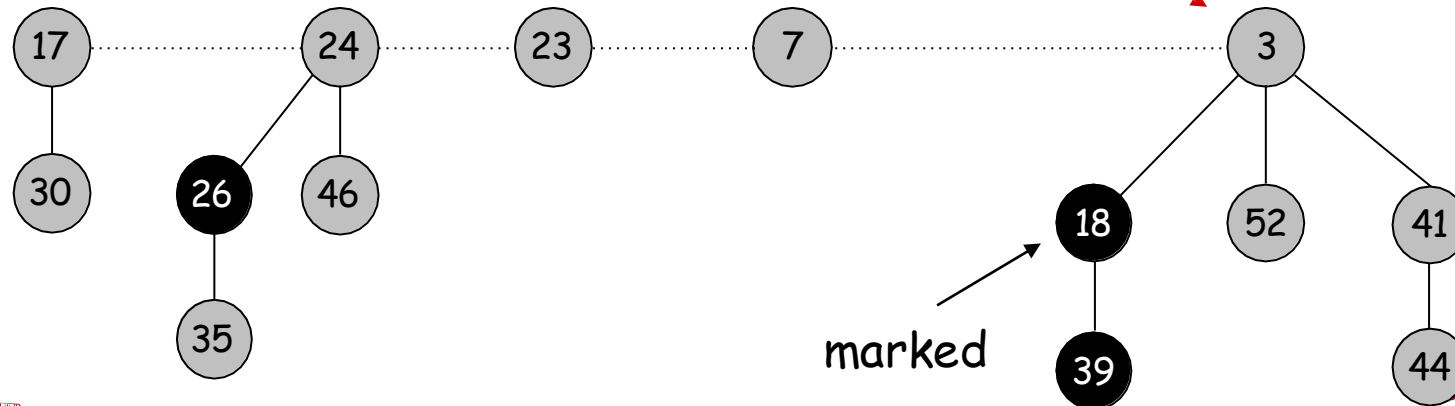
$n = 14$

$d = 3$

min

Heap H

marked





Fibonacci Heaps: Potential Function



may be scaled up by some constant sufficiently large

$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

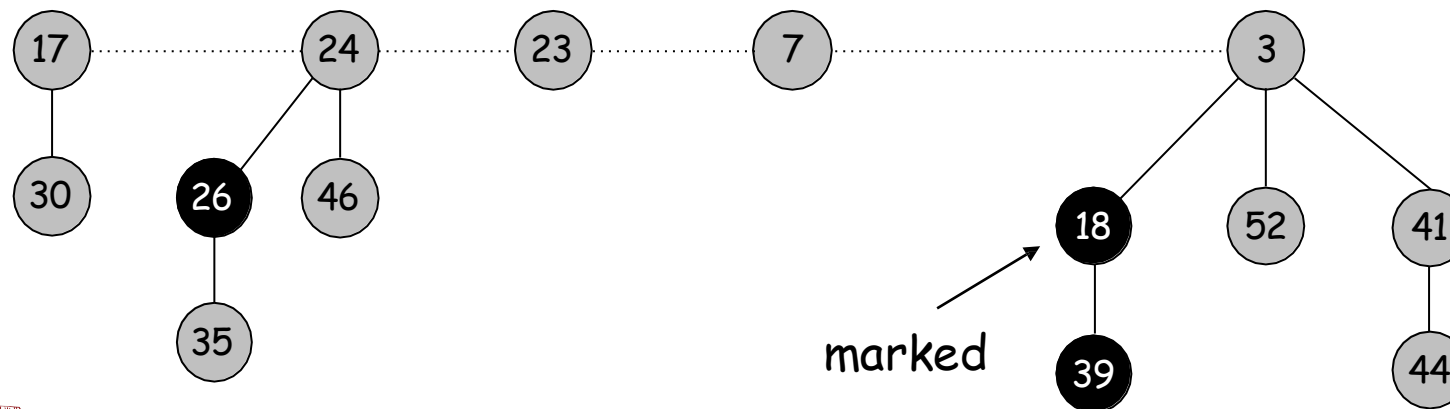
potential of heap H

$\text{trees}(H) = 5$

$\text{marks}(H) = 3$

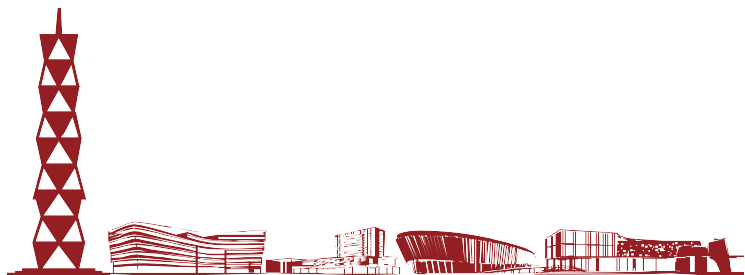
$\Phi(H) = 5 + 2 \cdot 3 = 11$

Heap H



Let H and H' denote the heap before and after an operation.

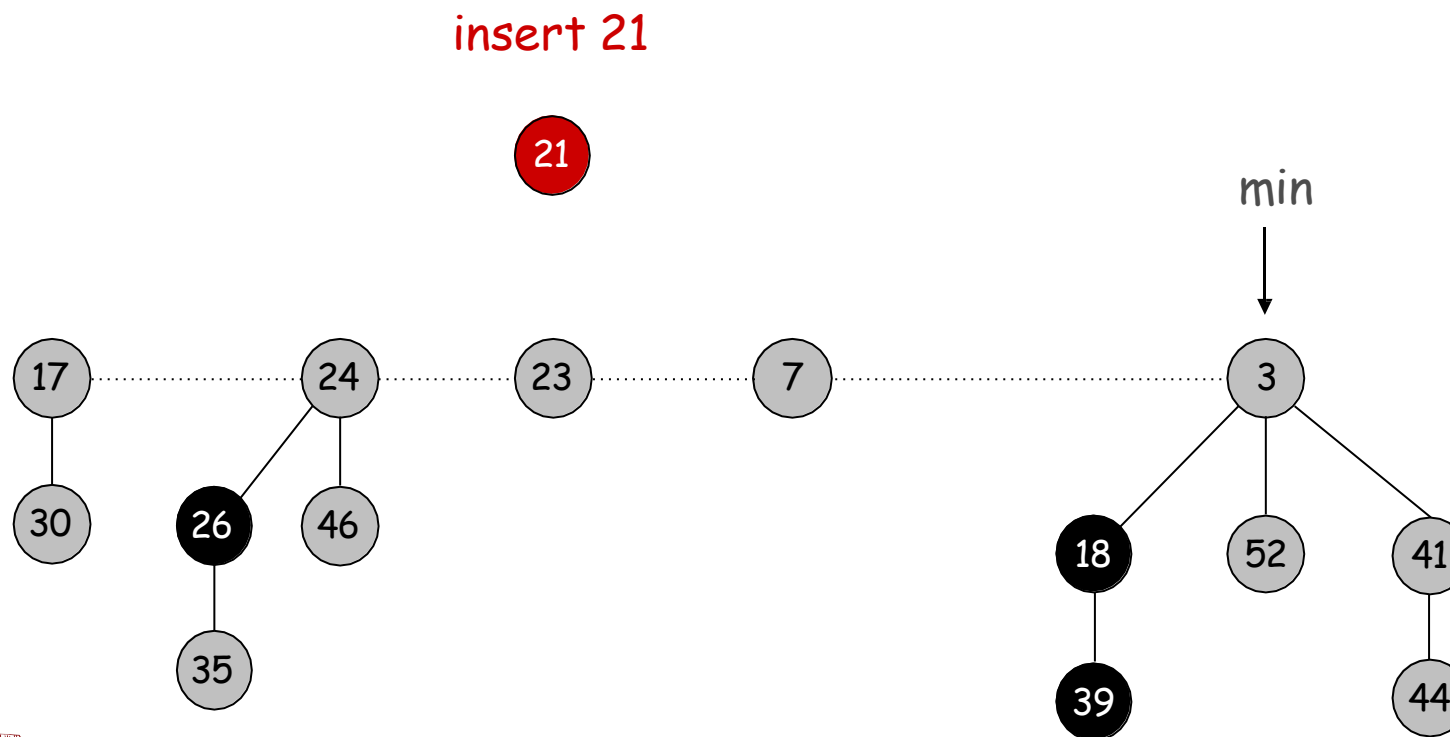
- **Make-Heap**
 - Make an empty heap. Set $H'.n=0$, $H'.min=NIL$
 - Cost = $O(1)$
 - Amortized cost = $O(1)$, since $\Phi(H) = \Phi(H') = 0$
- ▣ **Find the minimum**
 - Return $H.min$
 - Cost = amortized cost = $O(1)$





Insert.

- Create a new singleton tree.
- Add to root list; update min pointer (if necessary).



立志成才报国裕民



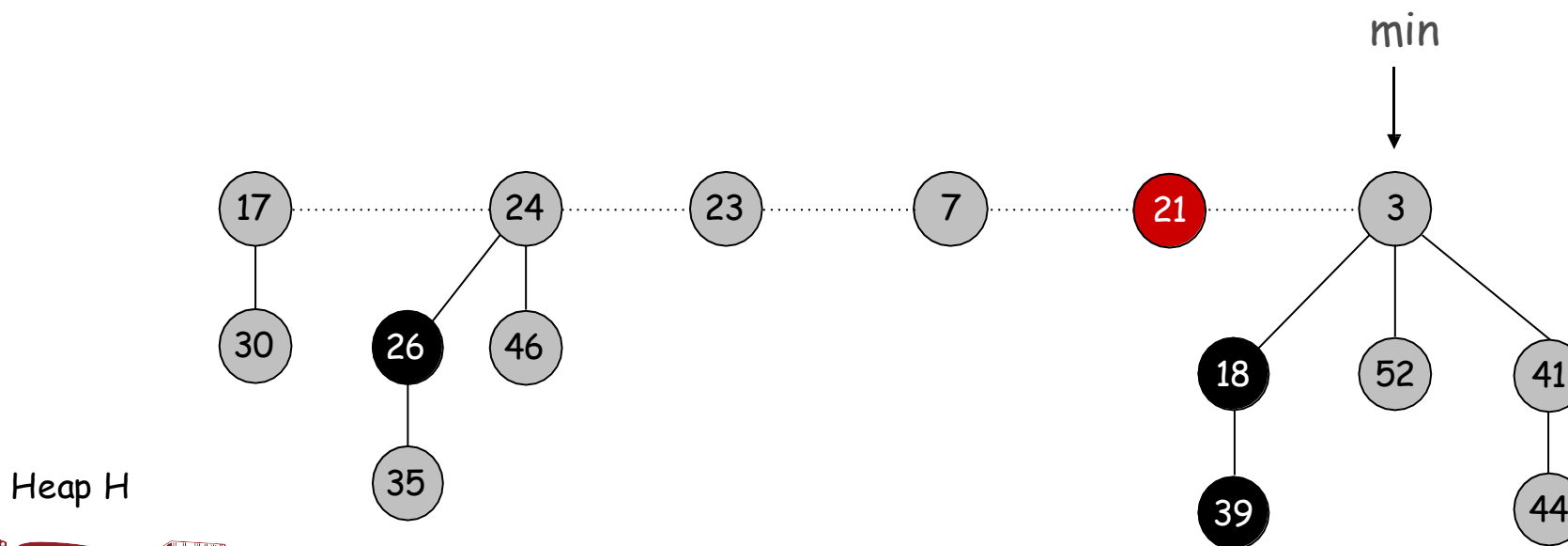
Fibonacci Heaps: Insert



Insert.

- Create a new singleton tree.
- Add to root list; update min pointer (if necessary).

insert 21



Heap H





Fibonacci Heaps: Insert Analysis



Insert a node

- Add the new node to the root list, left of the min node.
- Change H.min if new node's key is smaller.
- Cost = $O(1)$. Amortized
- cost = $O(1)$.
 - Number of roots increases by 1.
 - So $\Phi(H') - \Phi(H) = (t(H) + 1 + 2m(H')) - (t(H) + 2m(H)) = O(1)$.

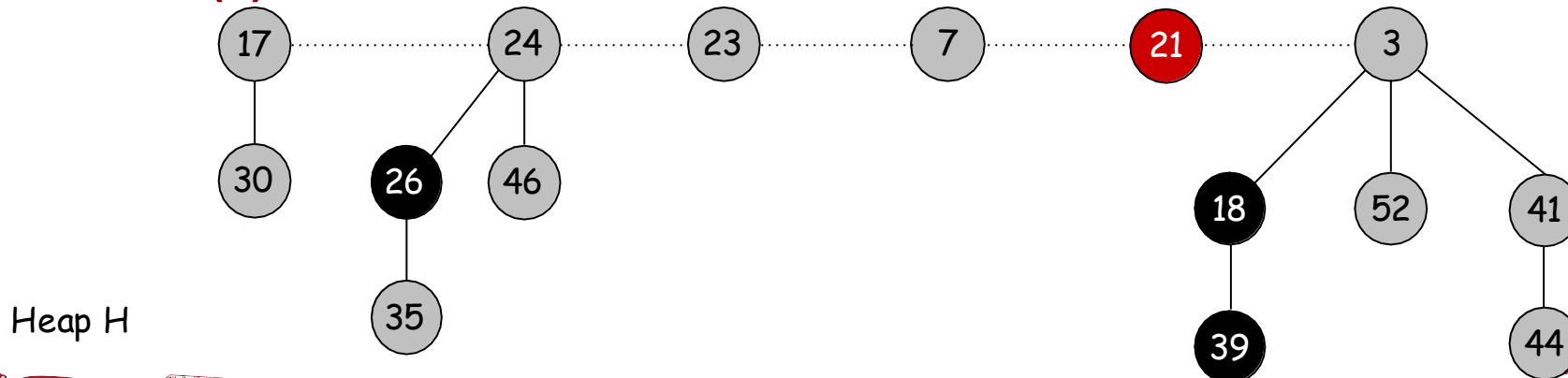
$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

potential of heap H

Actual cost $O(1)$

Change in potential. +1

Amortized cost $O(1)$



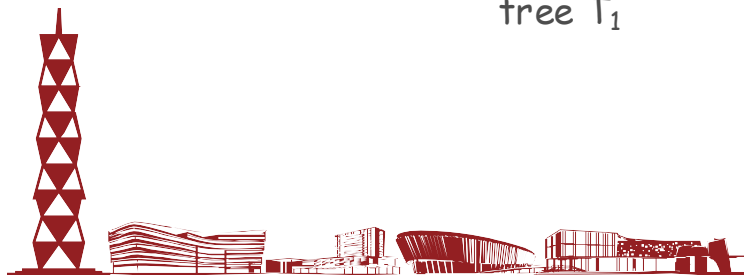
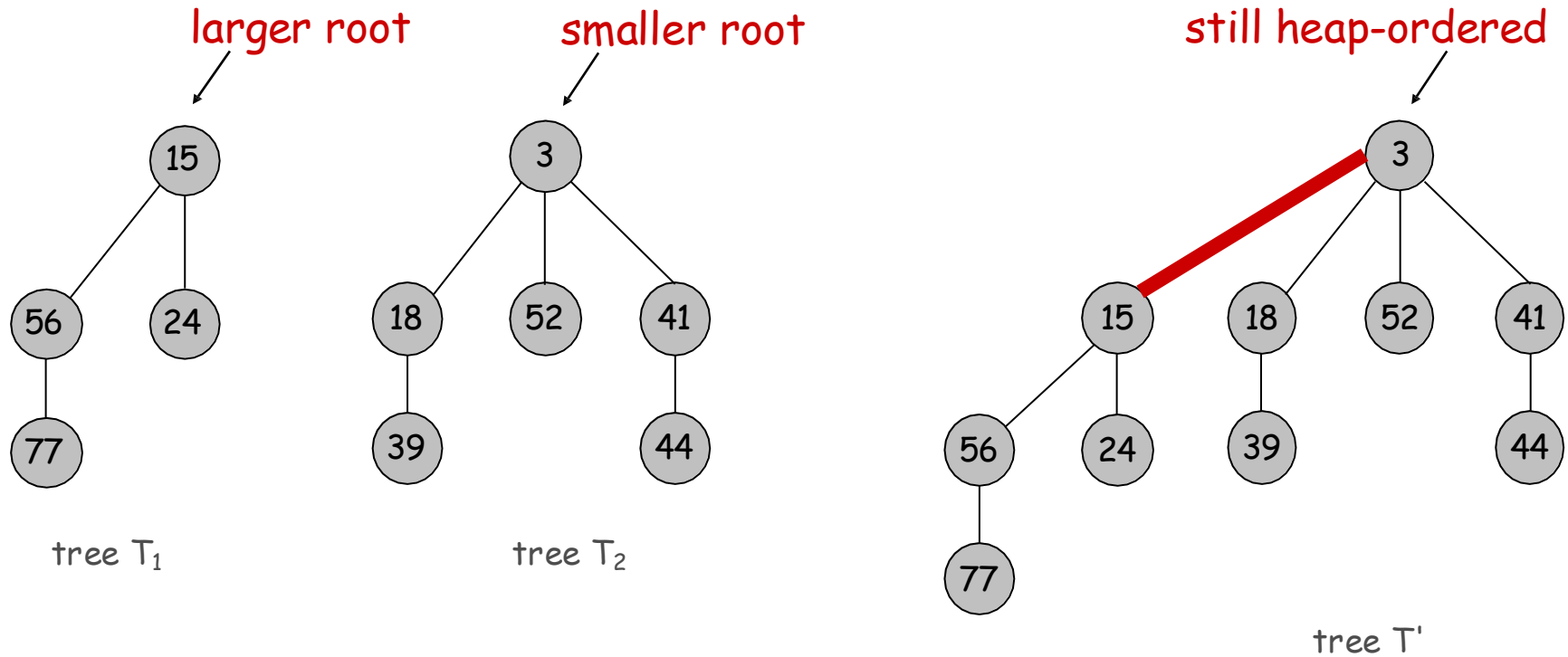
Heap H



Linking Operation



Linking operation. Make larger root a child of smaller root.





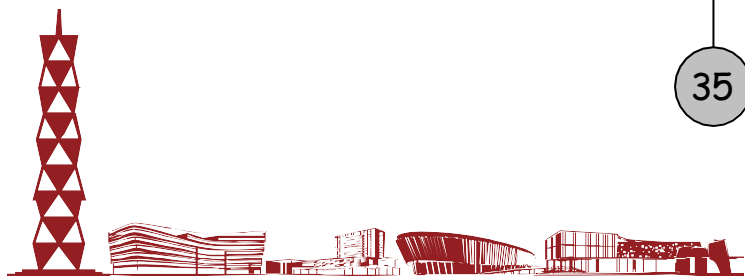
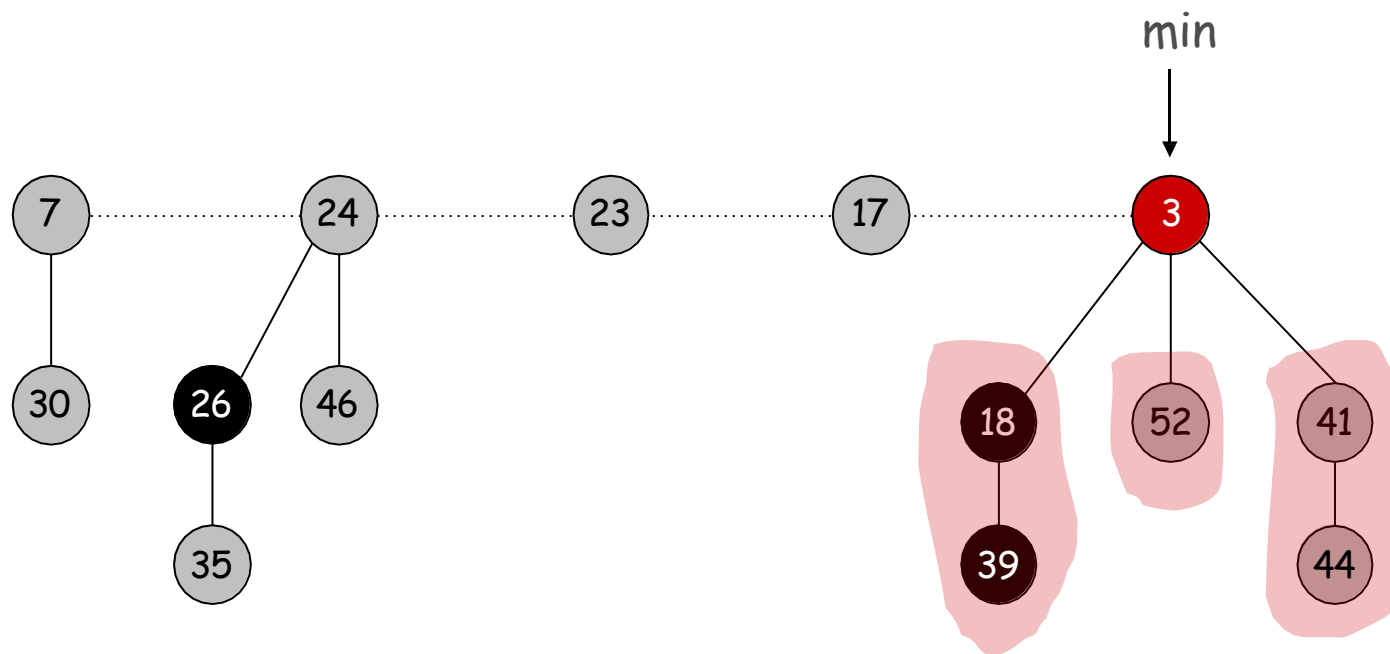
Fibonacci Heaps: Delete Min



Delete min.

Delete min; meld its children into root list; set unmarked; update min.

Consolidate trees so that no two roots have same degree.



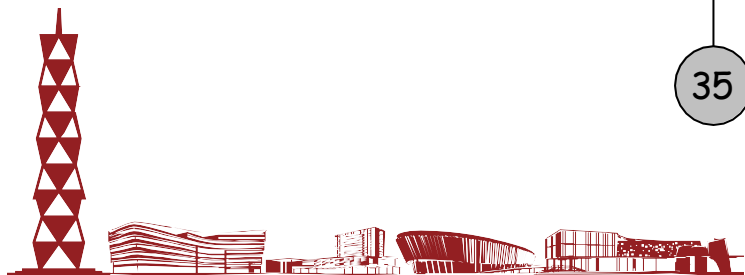
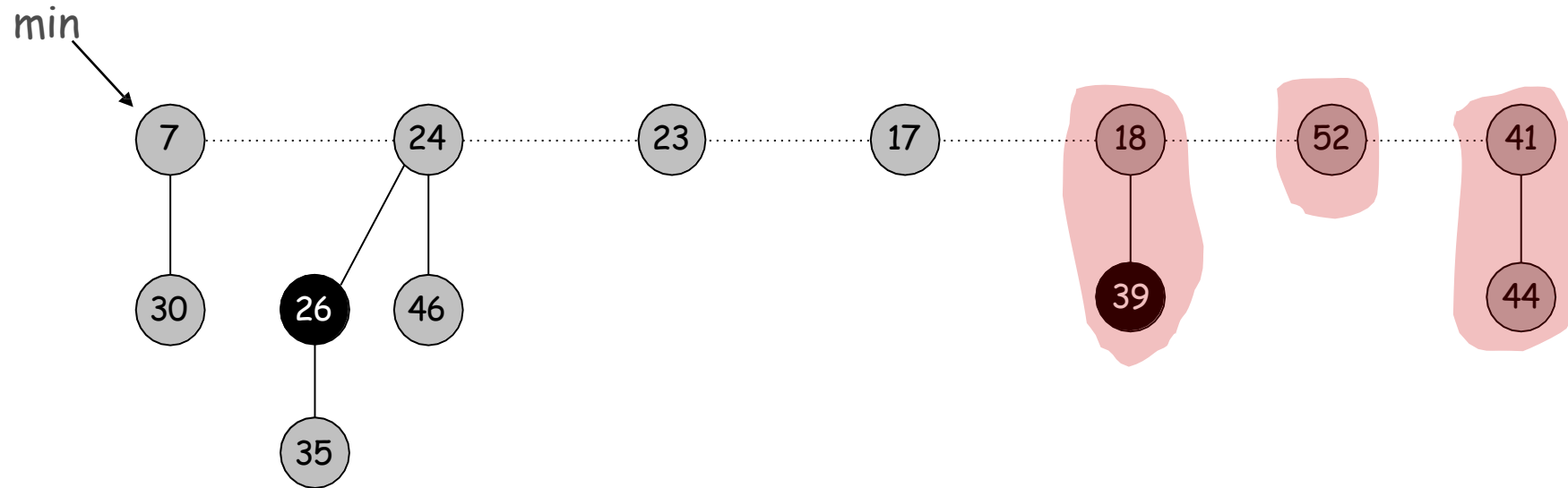


Fibonacci Heaps: Delete Min



Delete min.

Delete min; meld its children into root list; set unmarked; update min.
Consolidate trees so that no two roots have same degree.



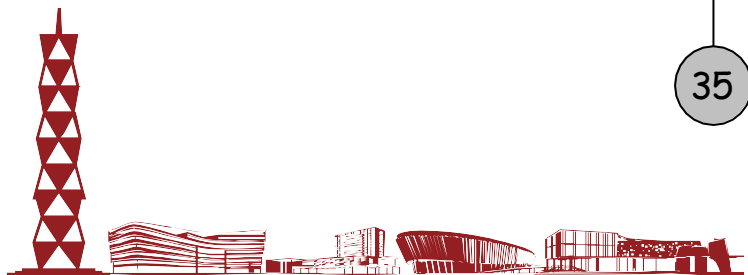
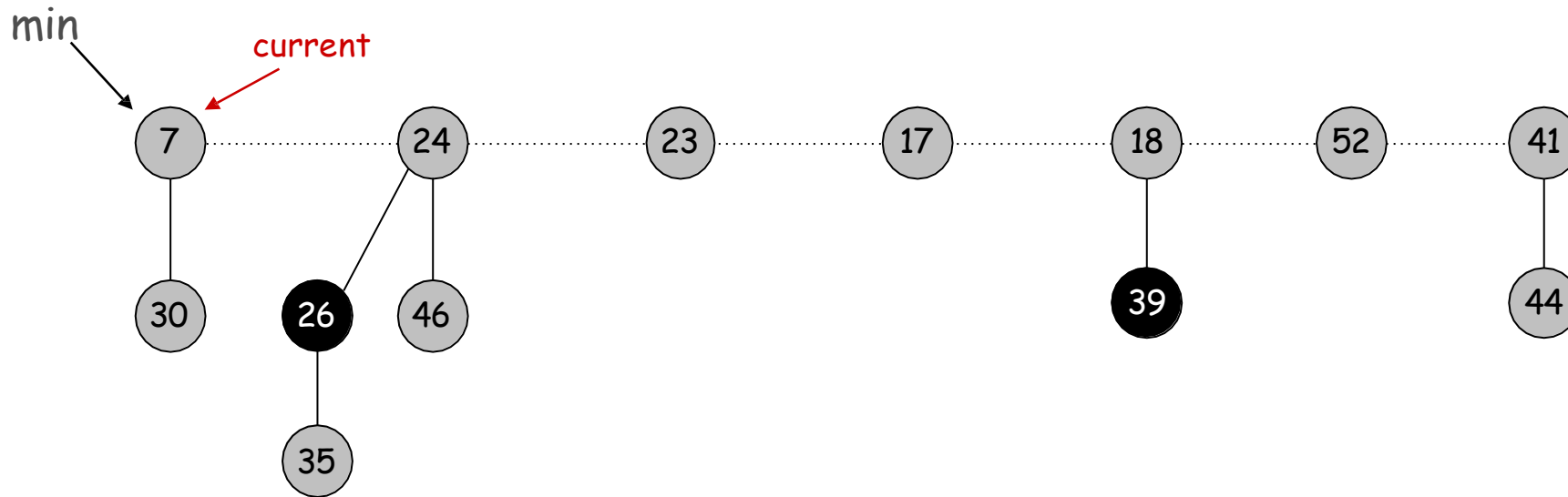


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



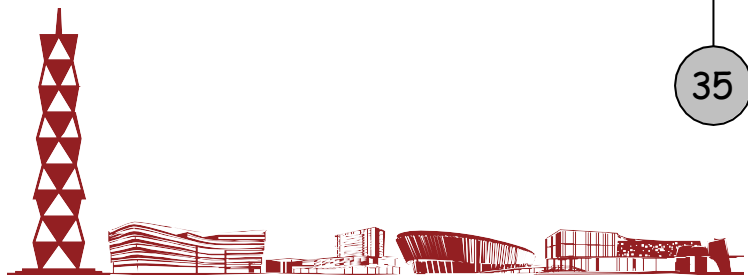
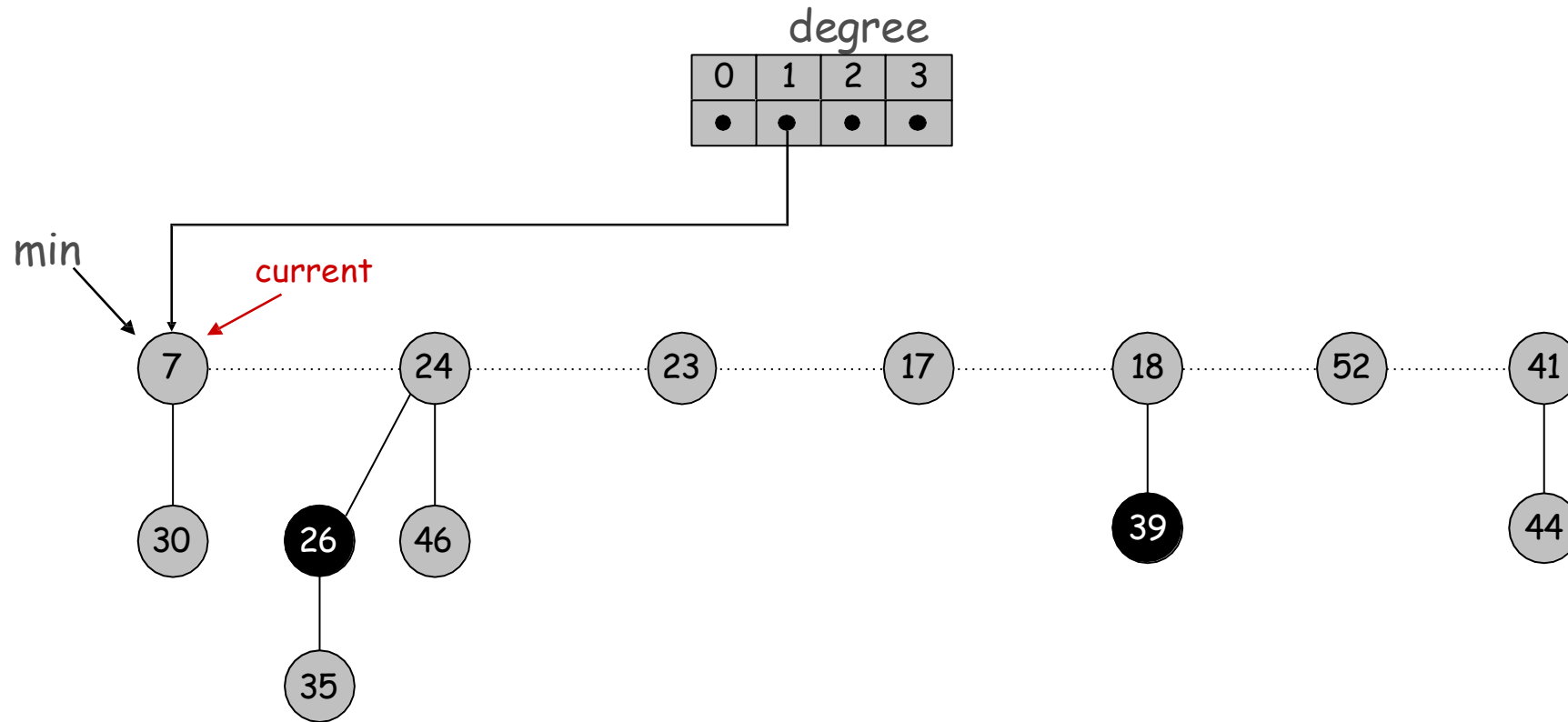


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



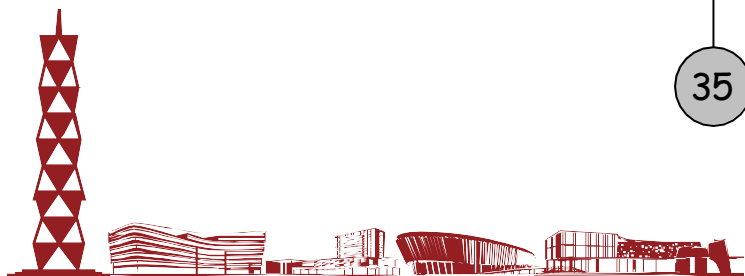
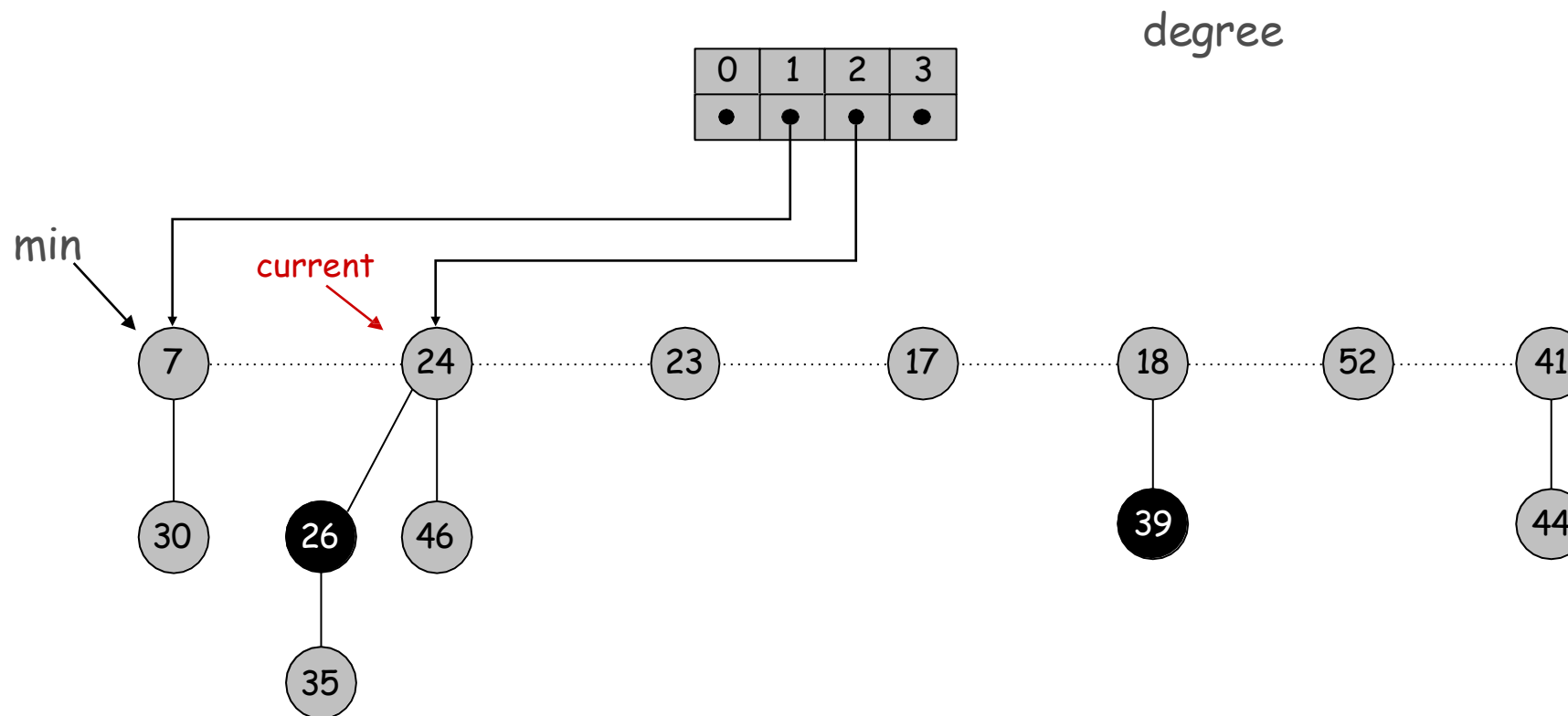


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



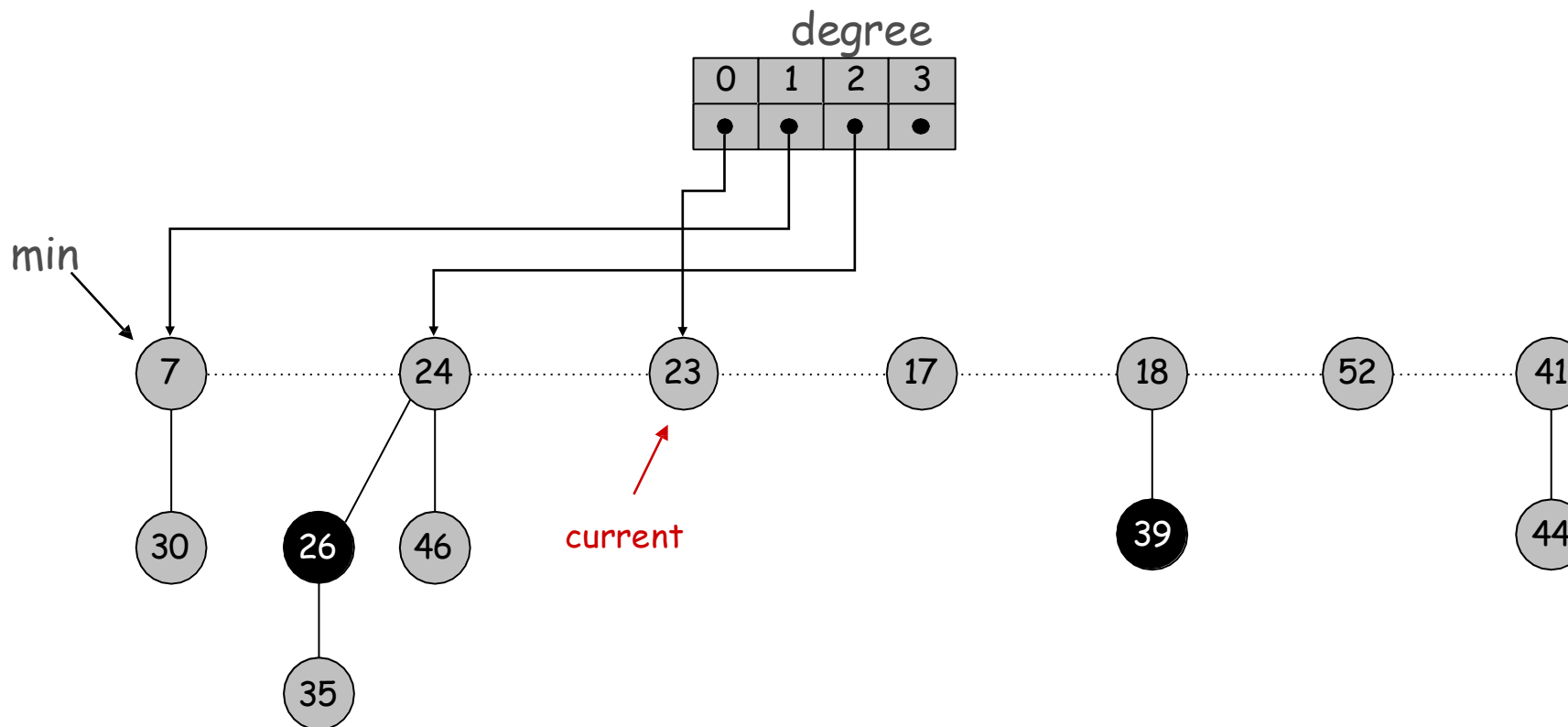


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



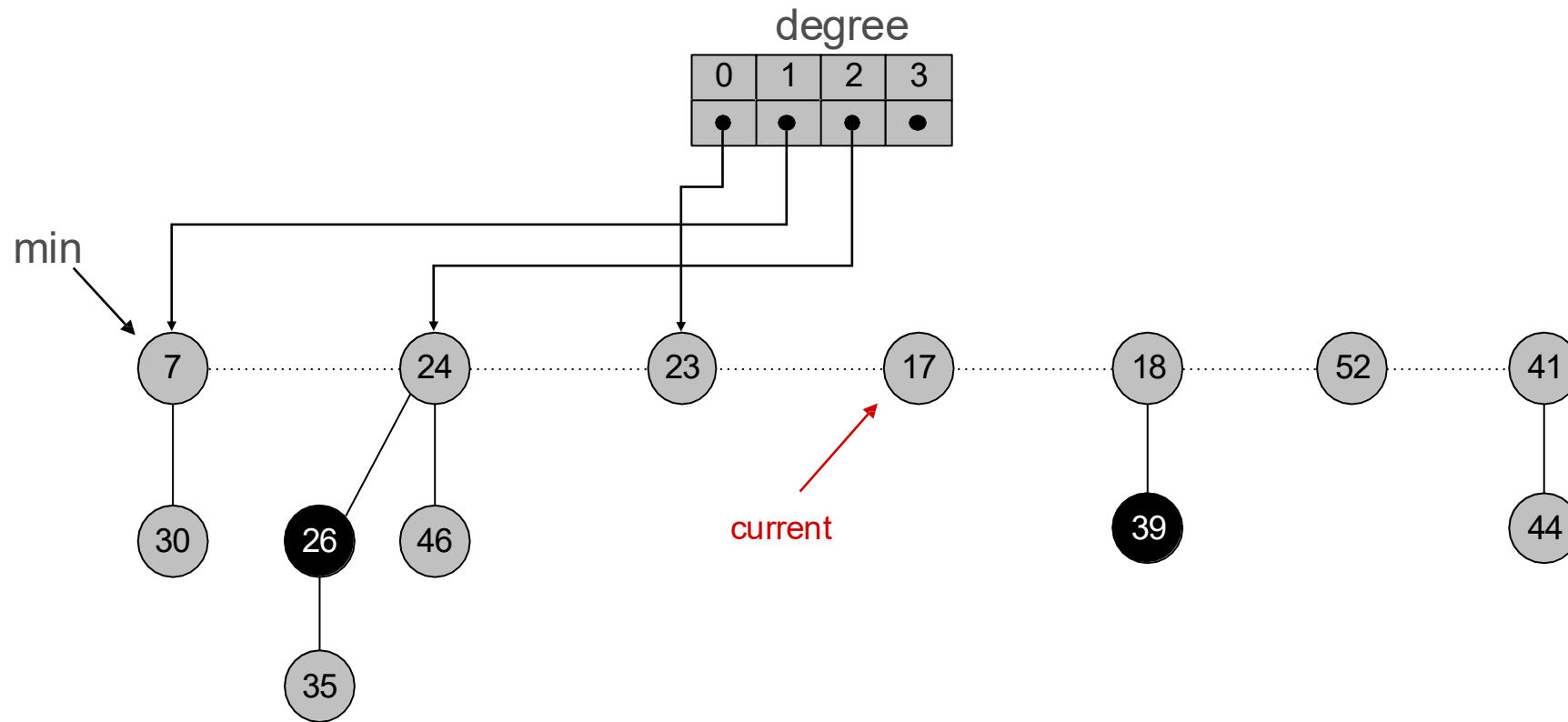


Fibonacci Heaps: Delete Min

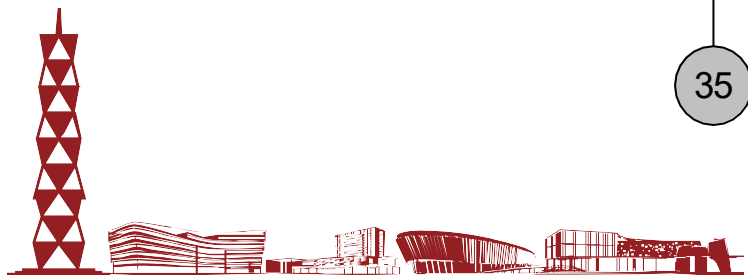


Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



link 23 into 17



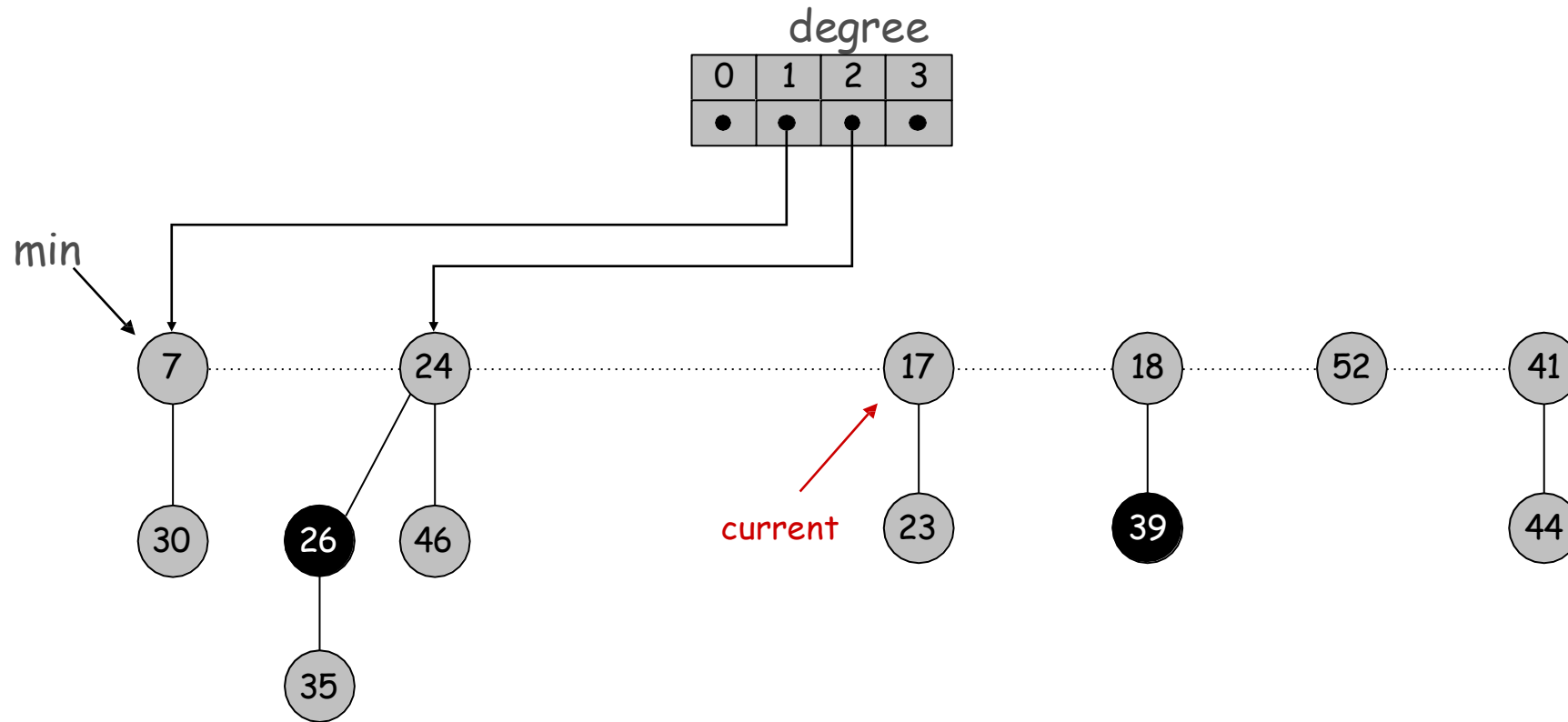


Fibonacci Heaps: Delete Min

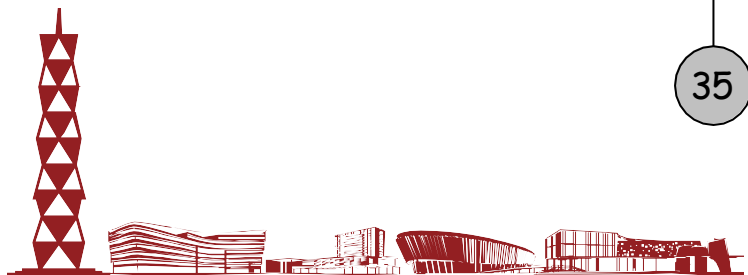


Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



link 17 into 7



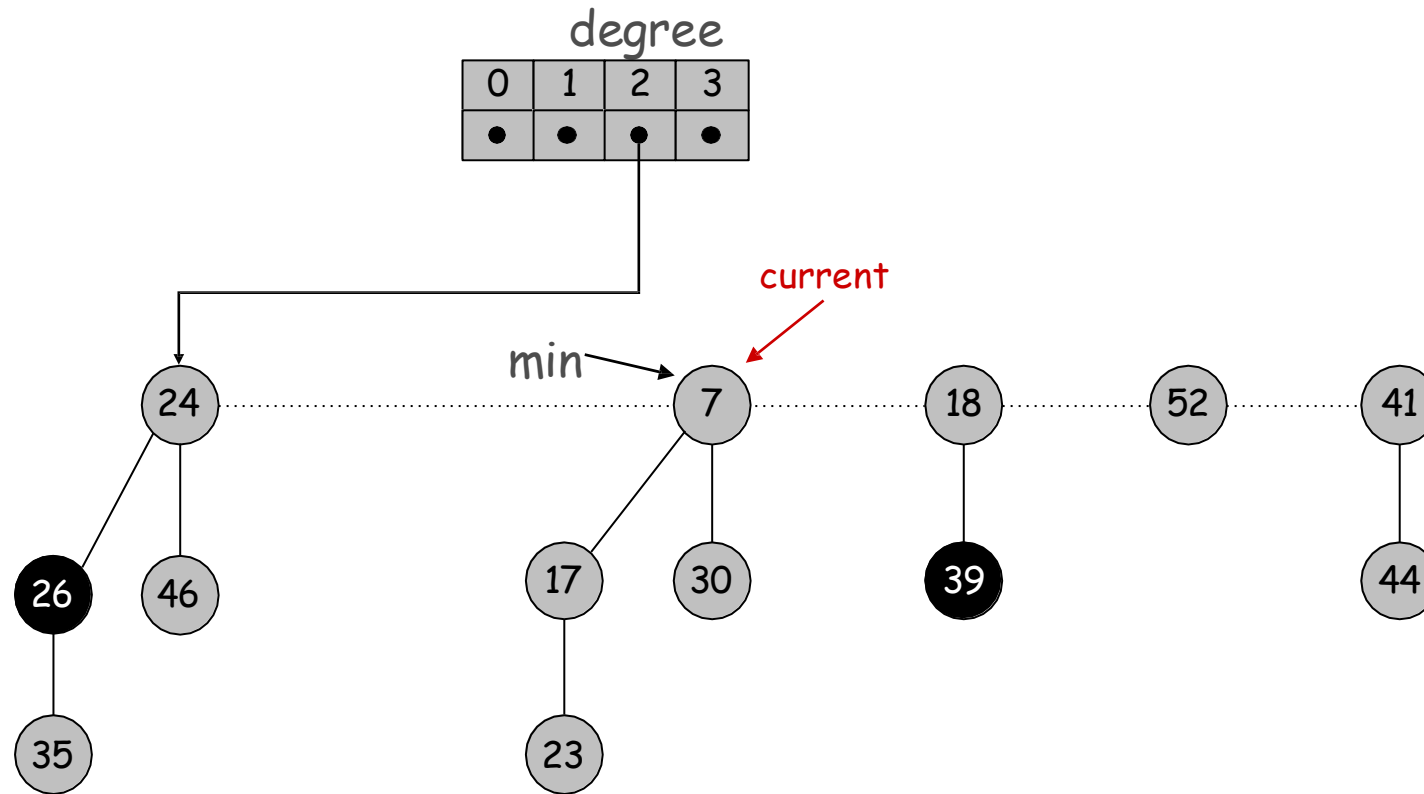


Fibonacci Heaps: Delete Min

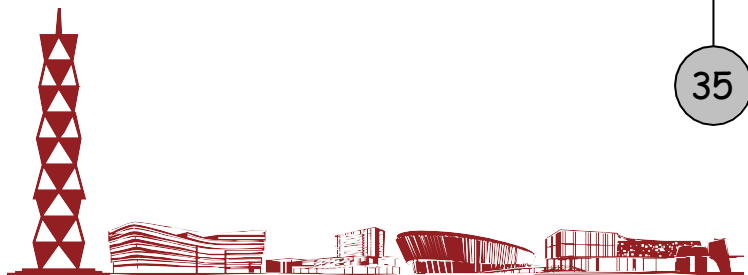


Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



link 24 into 7



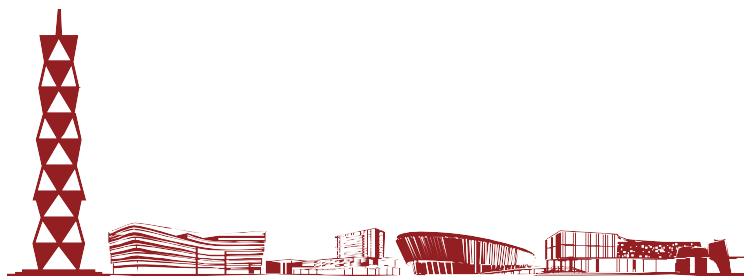
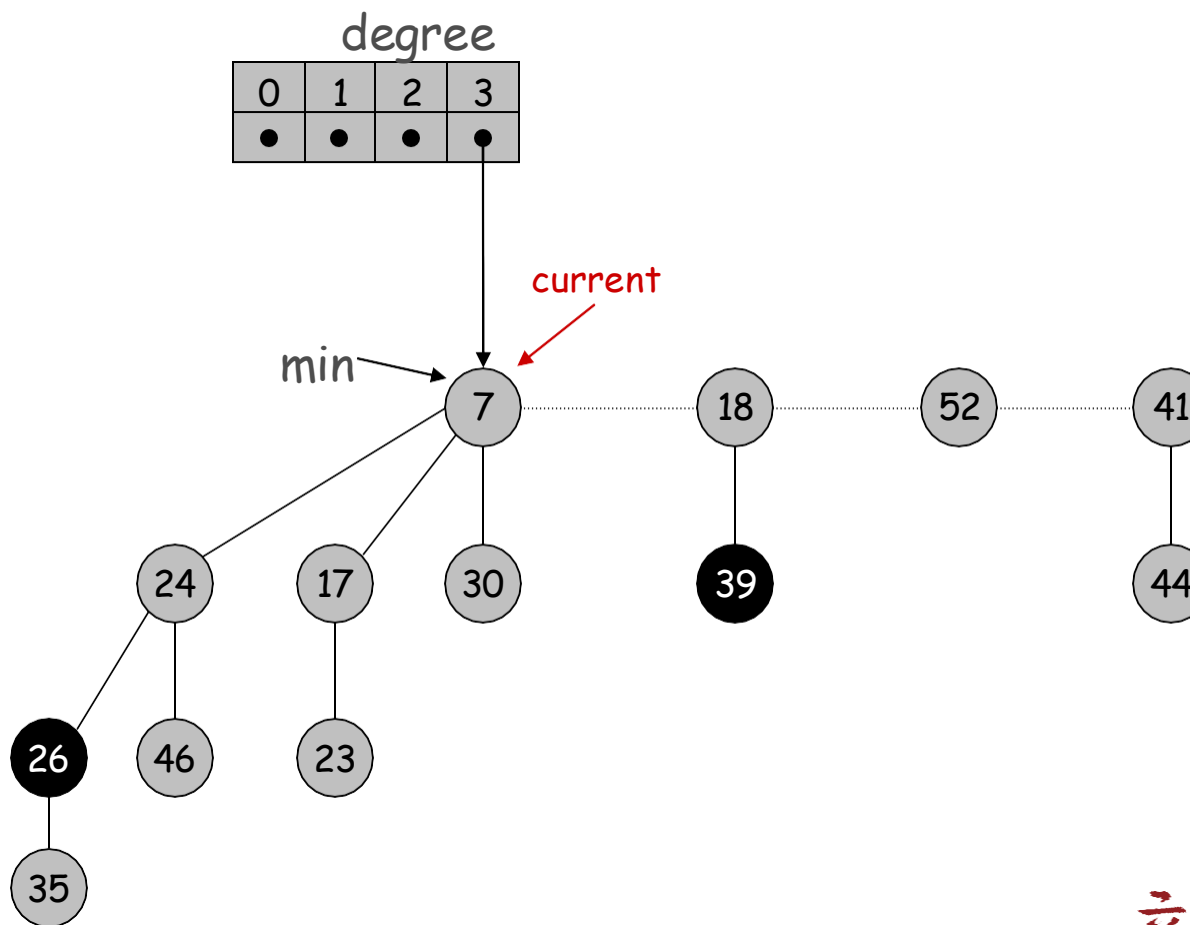


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



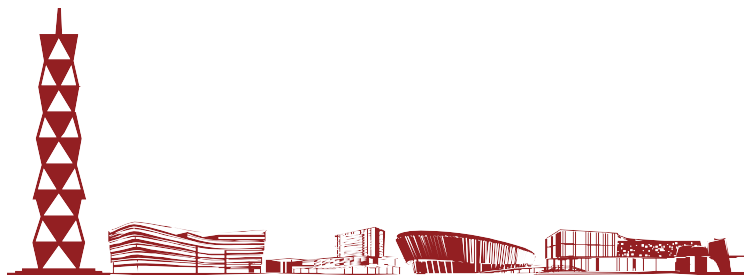
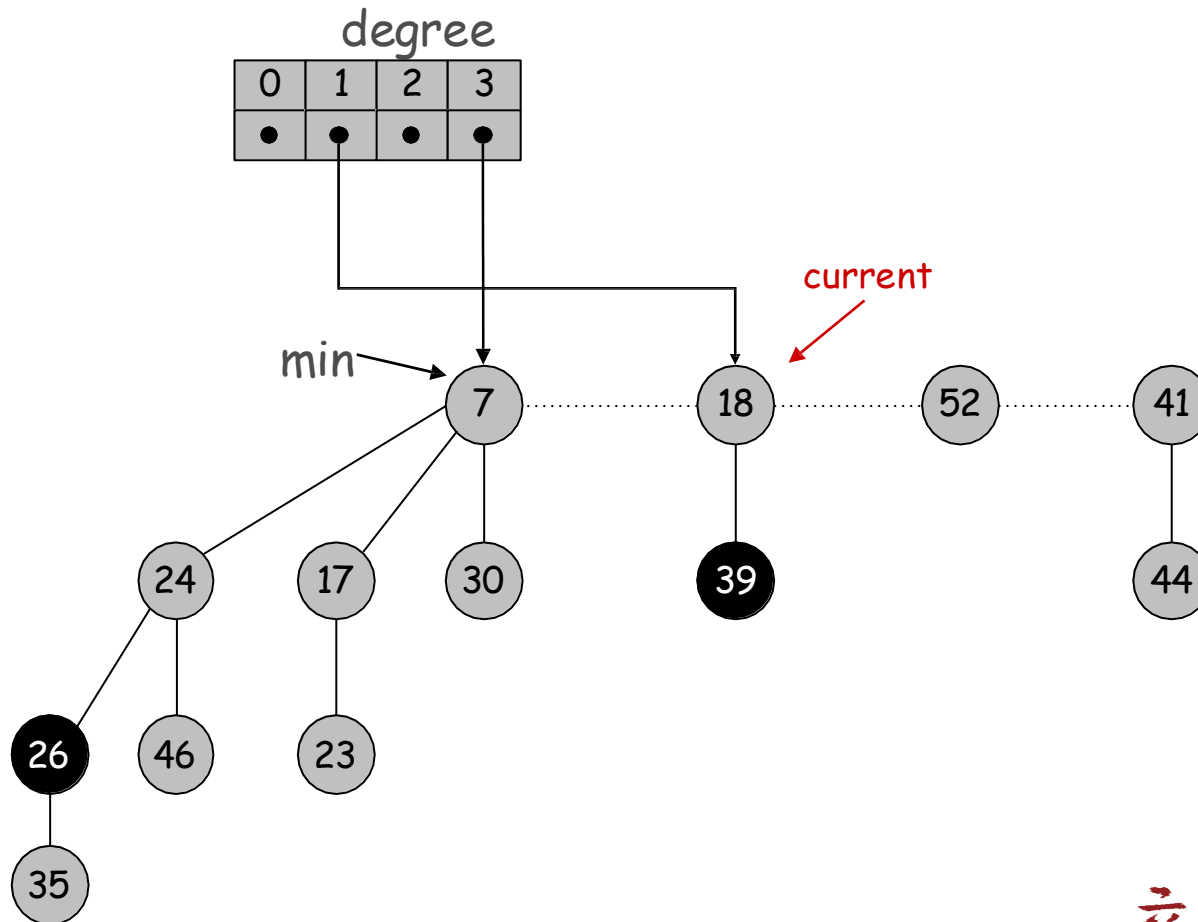


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



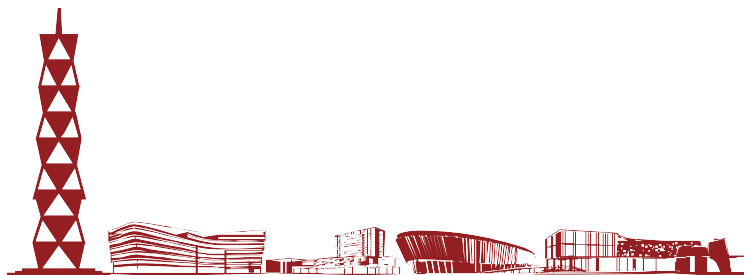
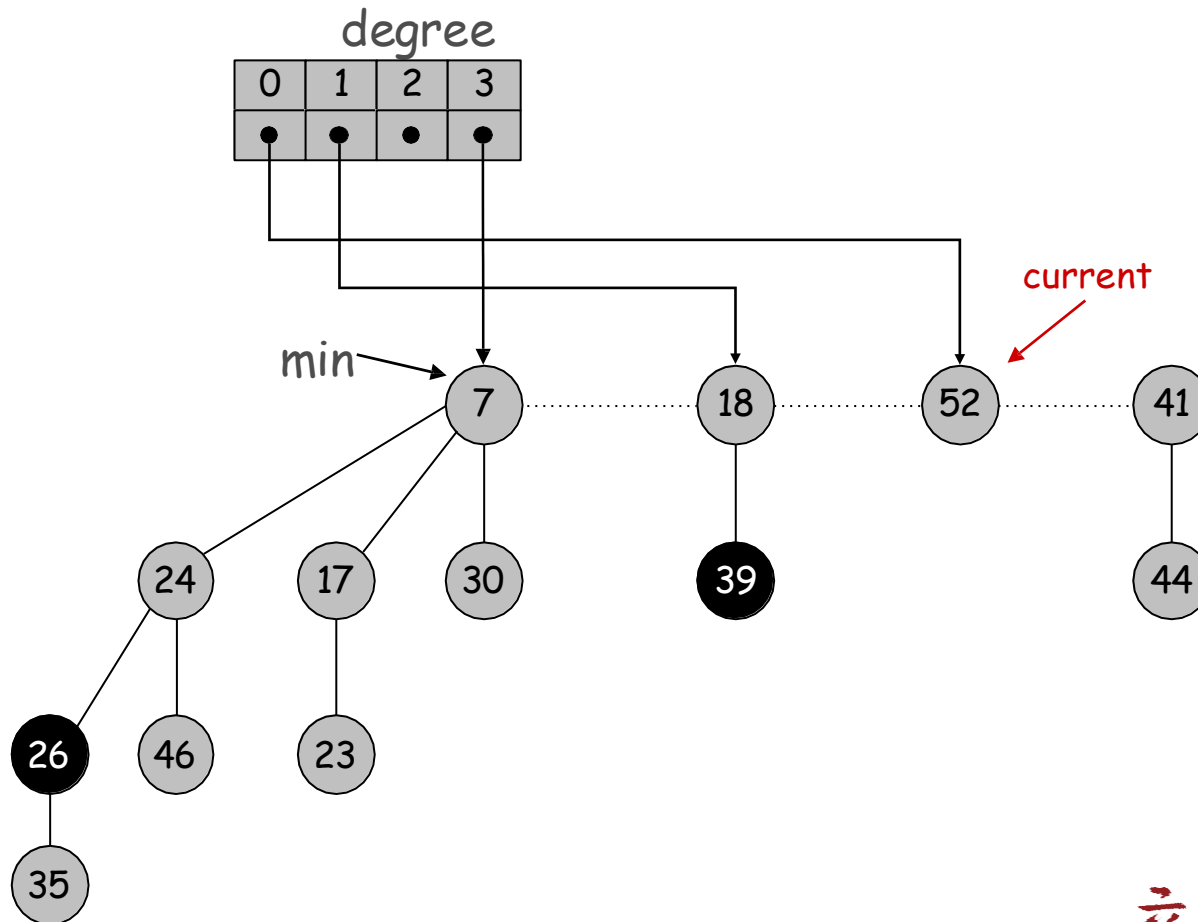


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



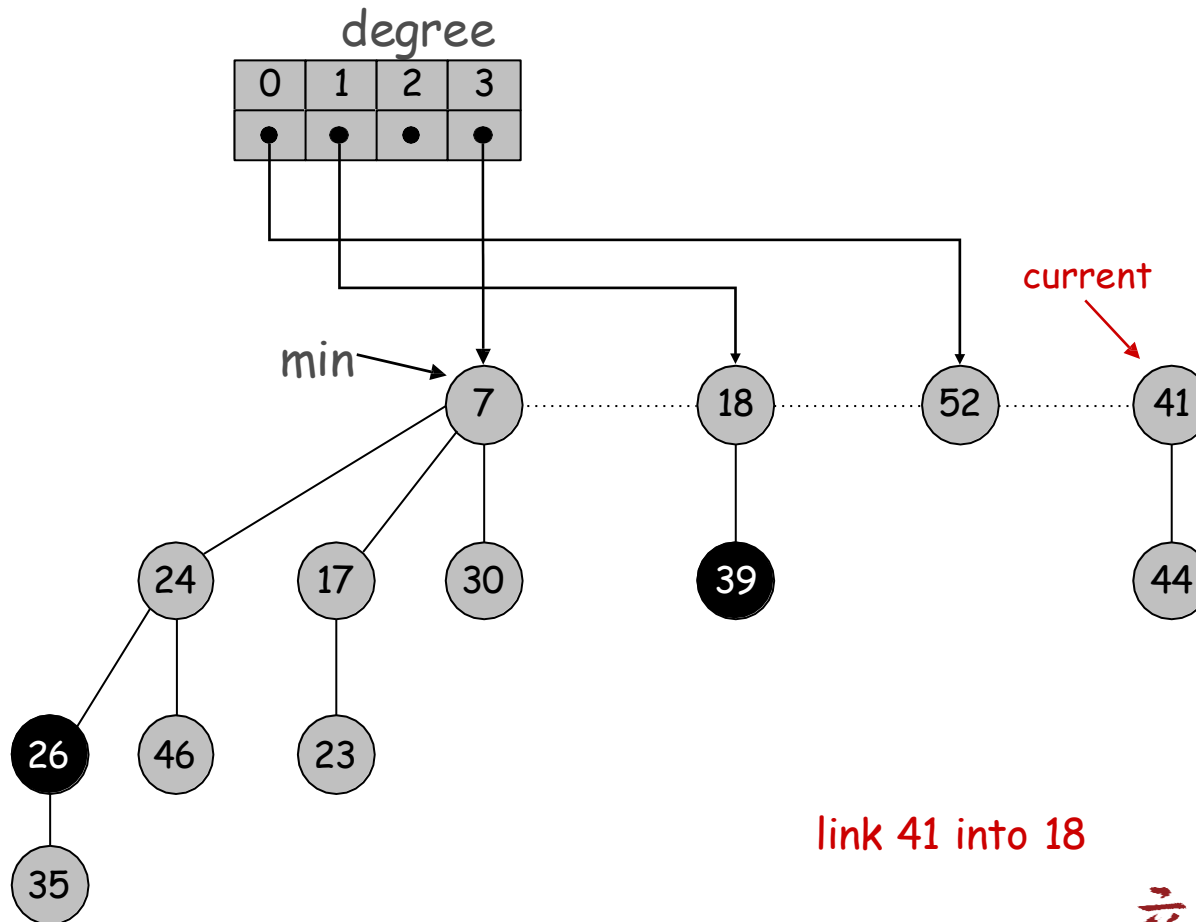


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



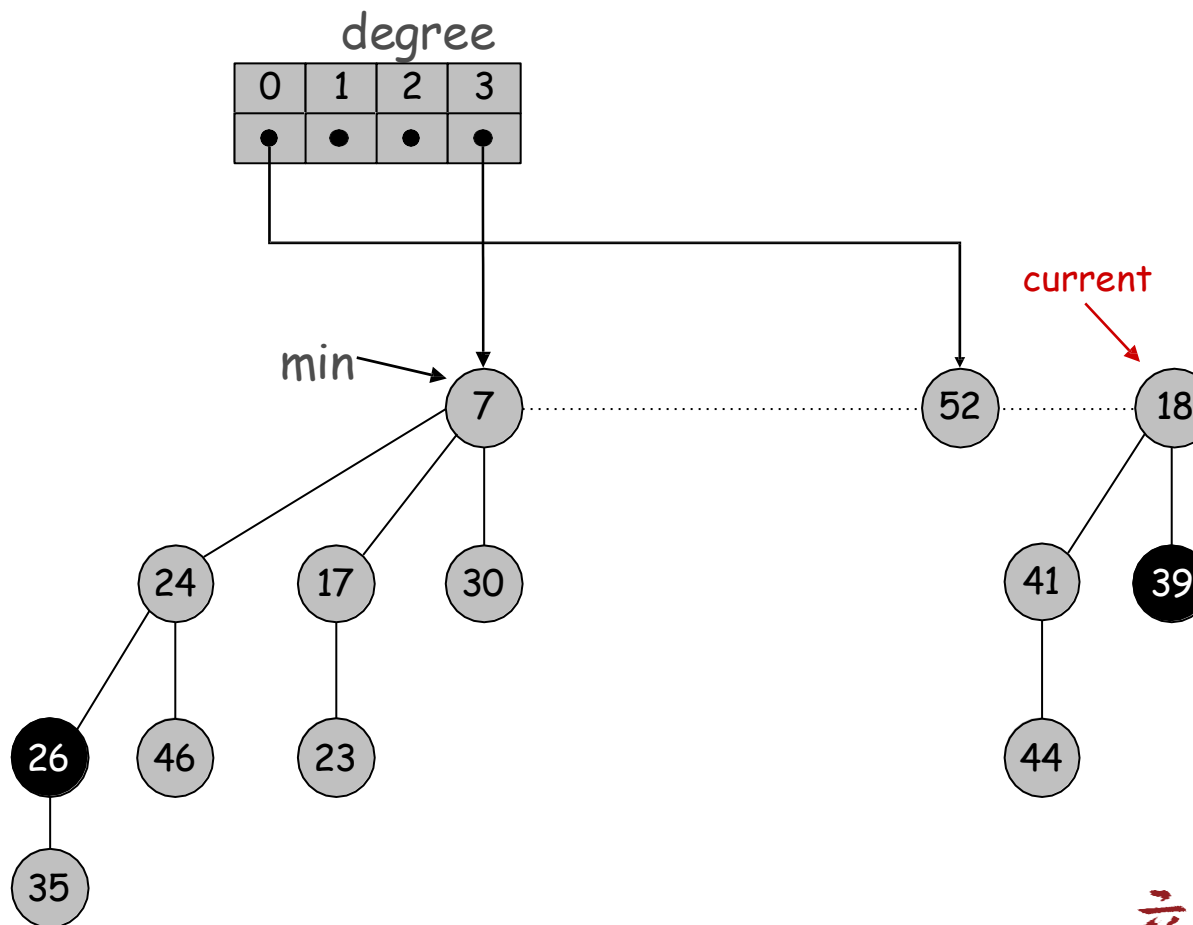


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



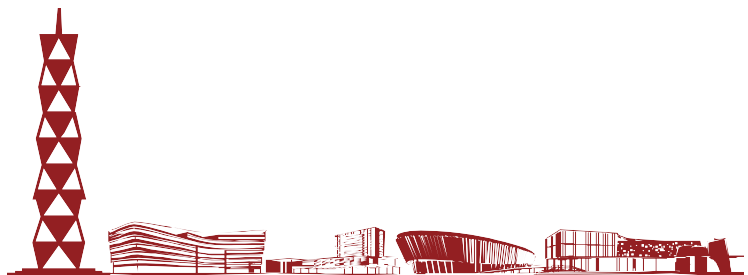
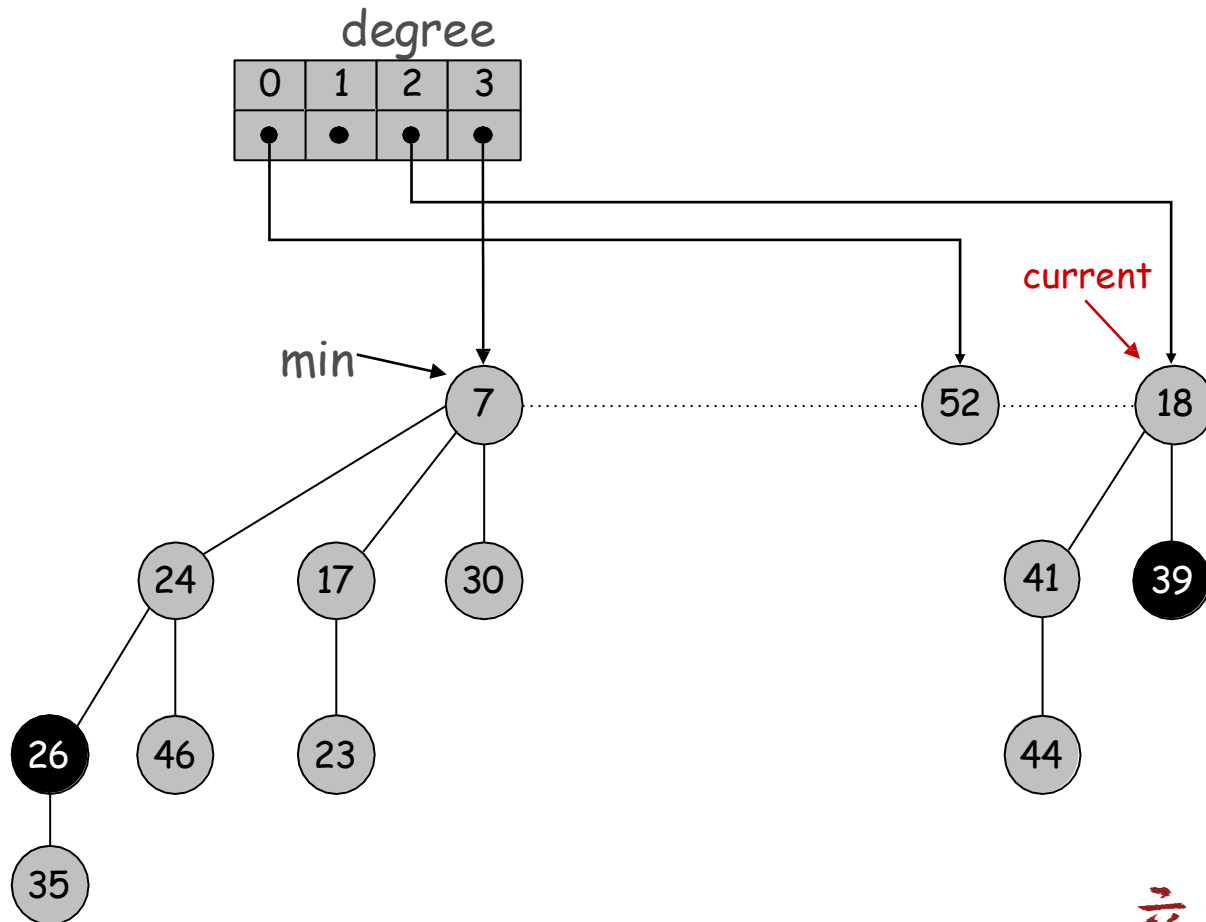


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.



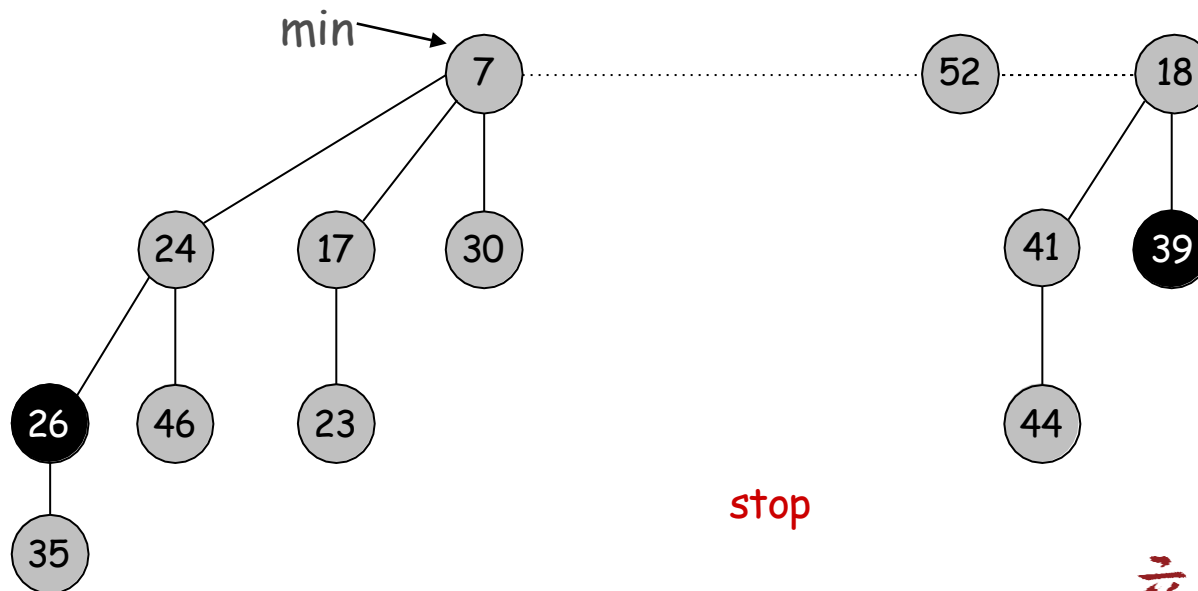


Fibonacci Heaps: Delete Min



Delete min.

- Delete min; meld its children into root list; set unmarked; update min.
- Consolidate trees so that no two roots have same degree.





Fibonacci Heaps: Delete Min



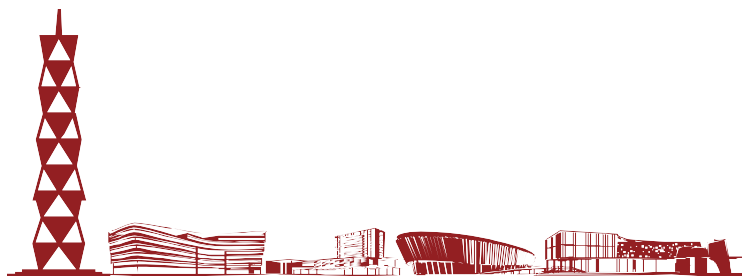
First remove the H.min node.

Then add each of its children (along with its subtree) to the root list.

- There may now be many trees in the root list.
- To find the new H.min, we need to iterate through the roots of all the trees, which may be slow. So we want to
- decrease the number of trees in the root list.
- **Def** The degree of a node is its number of children.

We merge some of the trees in the root list, so that none of the roots have the same degree.

- The merging function is called CONSOLIDATE.





Fibonacci Heaps: Delete Min



Let $D(H.n)$ be upper bound on the degree of any node in a Fibonacci heap with n nodes.

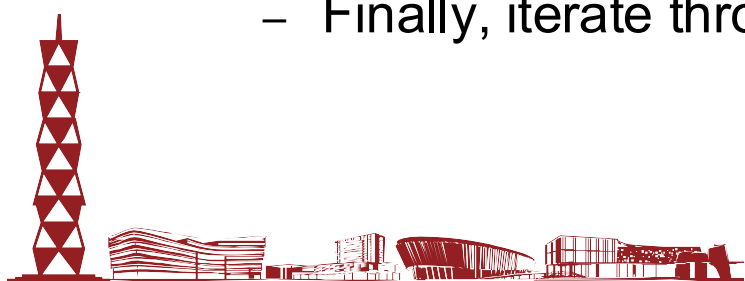
- We show later $D(H.n) = O(\log n)$.

Use an array A of size $D(H.n)+1$.

- $A[i]$ points to a tree in the root list with degree i .

Iterate through all the trees in the root list.

- If the current tree x we process has degree d , and $A[d] = y \neq NIL$, then there's already a tree y in the root list with degree d .
- Since we don't want two trees in the root list with the same degree, we link the roots of x and y .
 - This creates a tree in the root list with degree $d+1$, and removes the tree with degree d .
 - The direction we link depends on which root has the smaller key.
- Then set $A[d] = NIL$, and set $A[d+1]$ to point to newly linked tree.
- - If the new root is marked, clear the mark.
 - Finally, iterate through A array, and set $H.min$ to the min root value.





Fibonacci Heaps: Delete Min Analysis



Let H denote the heap before the delete min.

The real cost includes

- $O(D(n))$ for moving children of $H.min$ to root list.
- TCONSOLIDATE operate on a list of size at most $D(n)+t(H)-1$.
- Every time through the while loop, we link two of the trees in the root list.
 - Each tree can be linked (to a tree whose root has a smaller key) at most once.
 - So, the total number of iterations of the while loop is at most the root list size, i.e., $O(D(n)+t(H))$.
- **So, the real cost is $O(D(n)+t(H))$.**

$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

potential function

For the amortized cost, the potential before the extract min is at most $t(H)+2m(H)$.

The potential after extract is $\leq (D(n)+1)+2m(H)$.

- All trees in root list of H' have different degrees, and max degree is $D(n)$.
- No new nodes get marked during extract.

So amortized cost is

$$O(D(n) + t(H)) + ((D(n) + 1) + 2m(H)) - (t(H) + 2m(H)) = O(D(n)) + O(t(H)) - t(H) = O(D(n)).$$

- The last equality follows because we can scale up the units of the potential to cancel out the hidden constant in $O(t(H))$.



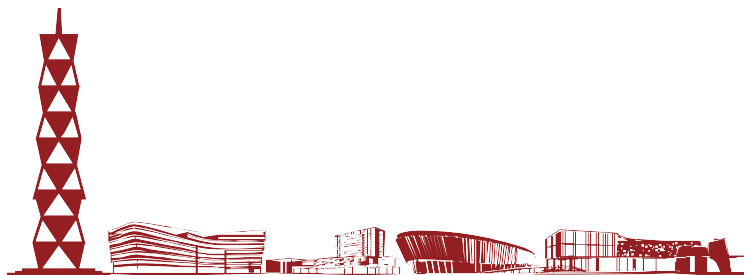
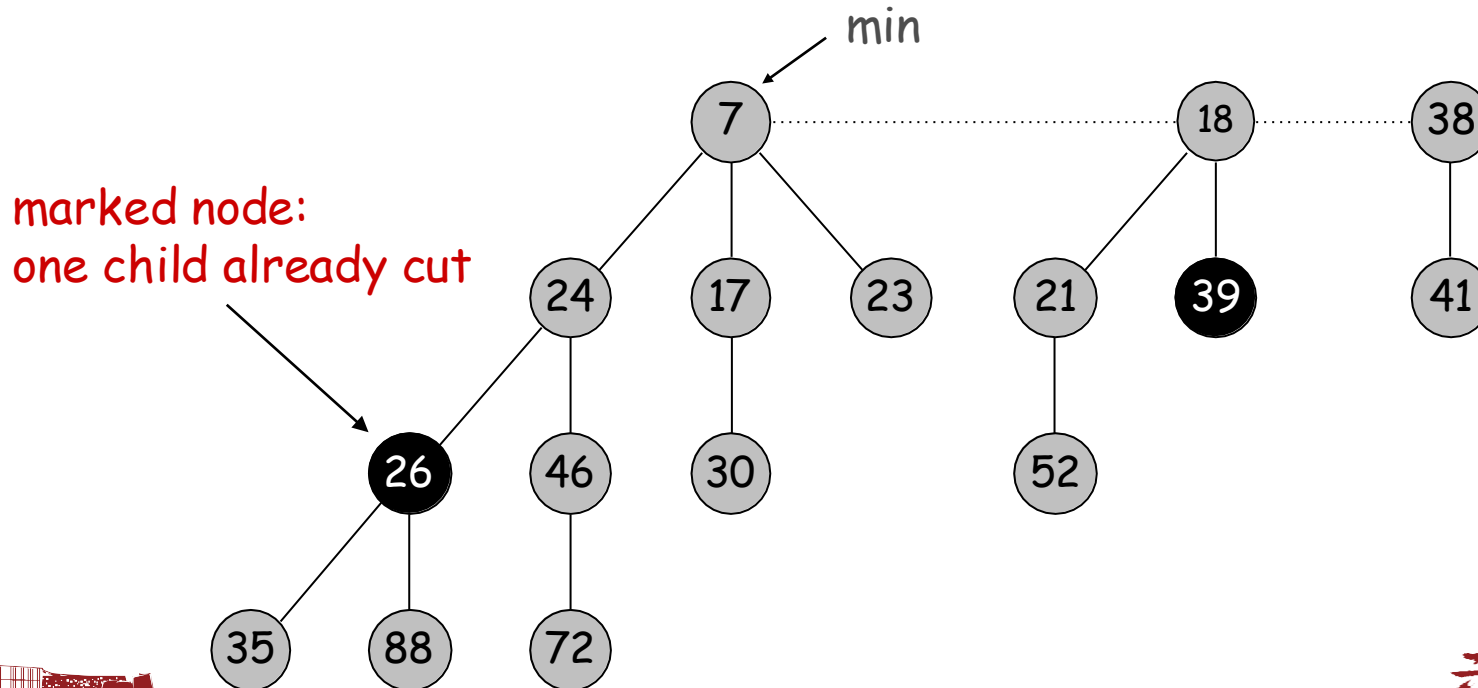


Fibonacci Heaps: Decrease Key



Intuition for decreasing the key of node x .

- If heap-order is not violated, just decrease the key of x .
- Otherwise, cut tree rooted at x , meld into root list, and unmark it.
- Look at p , the parent of x
 - If p is a root, stop
 - If it's unmarked, mark it
 - If it's marked, cut it off, meld into root list, and unmark it, and check the parent of p recursively.



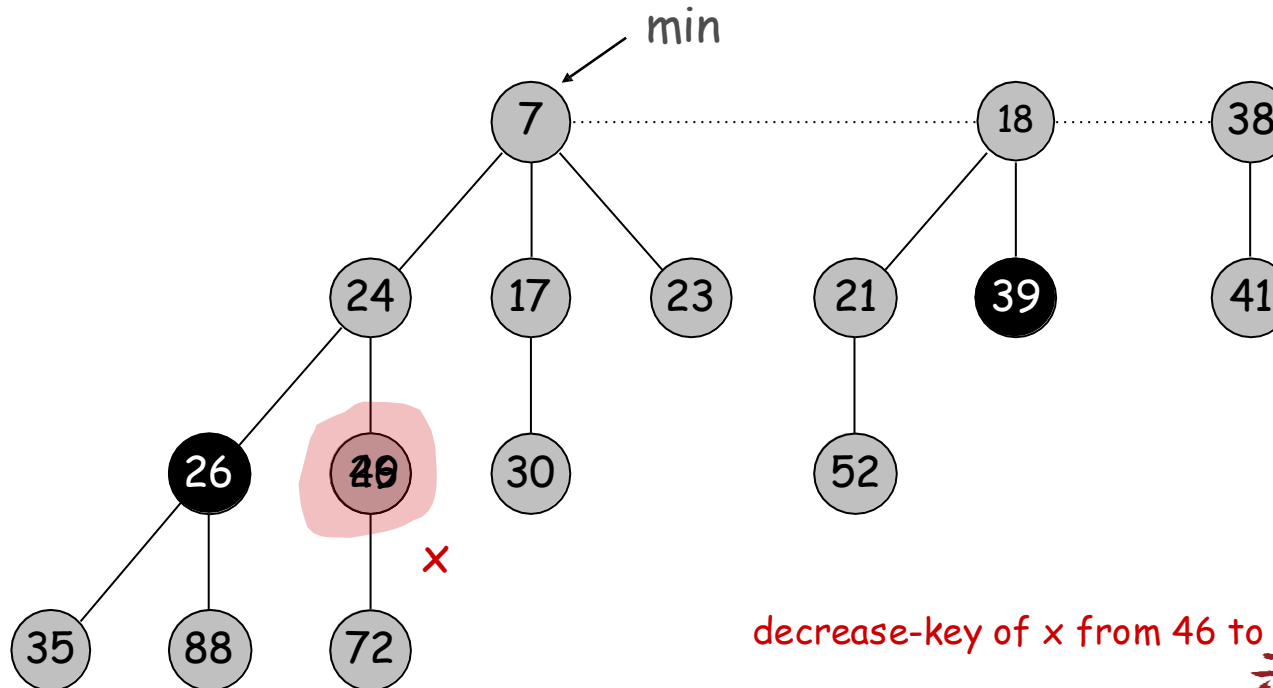


Fibonacci Heaps: Decrease Key



Case 1. [heap order not violated]

- Decrease key of x.
- Change heap min pointer (if necessary).



decrease-key of x from 46 to 29

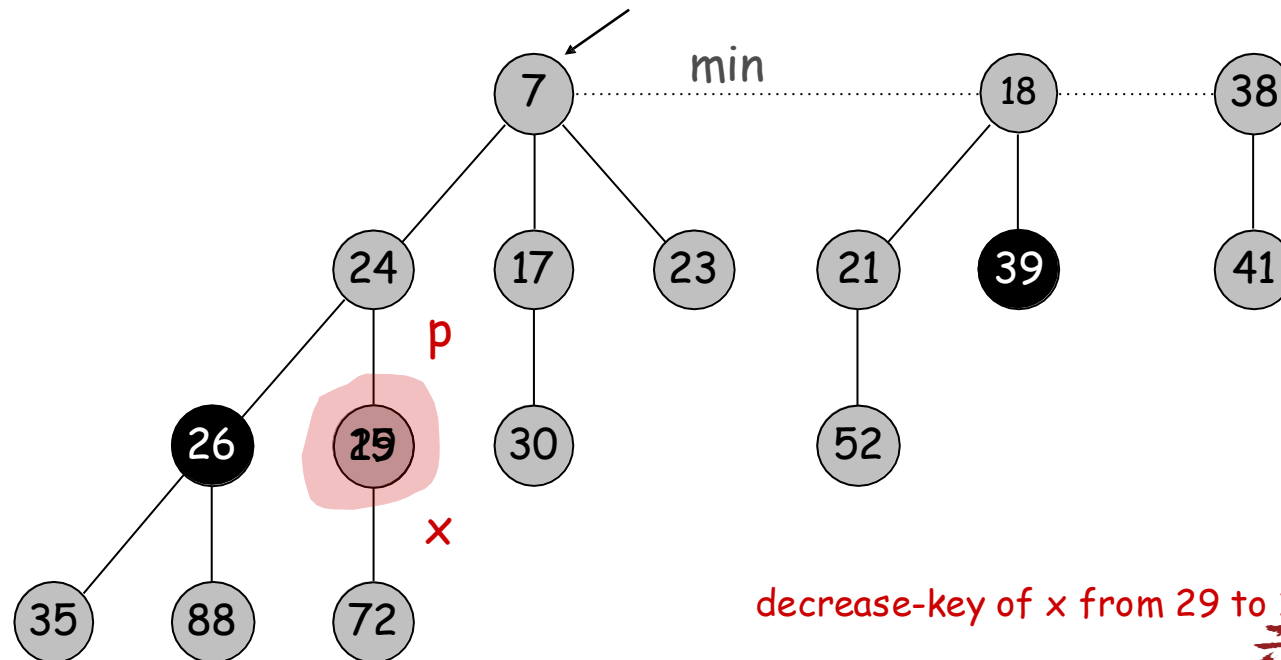


Fibonacci Heaps: Decrease Key



Case 2a. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



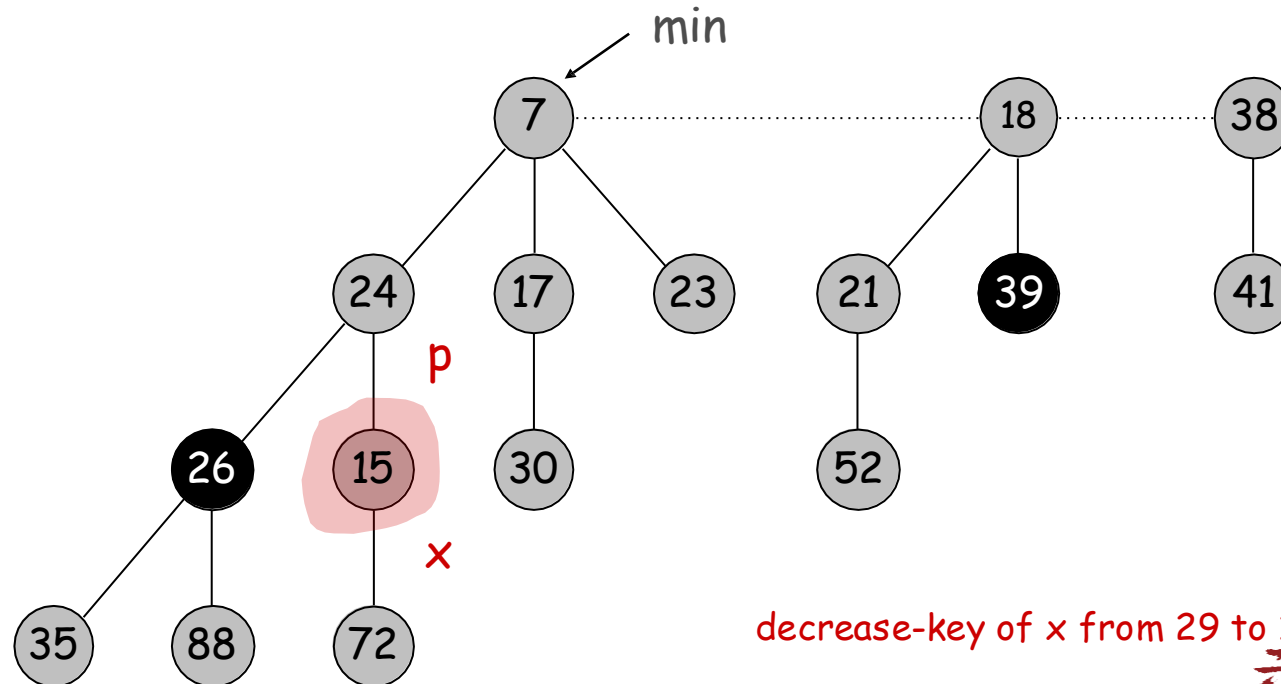


Fibonacci Heaps: Decrease Key



Case 2a. [heap order violated]

- Decrease key of x.
- Cut tree rooted at x, meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p, meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



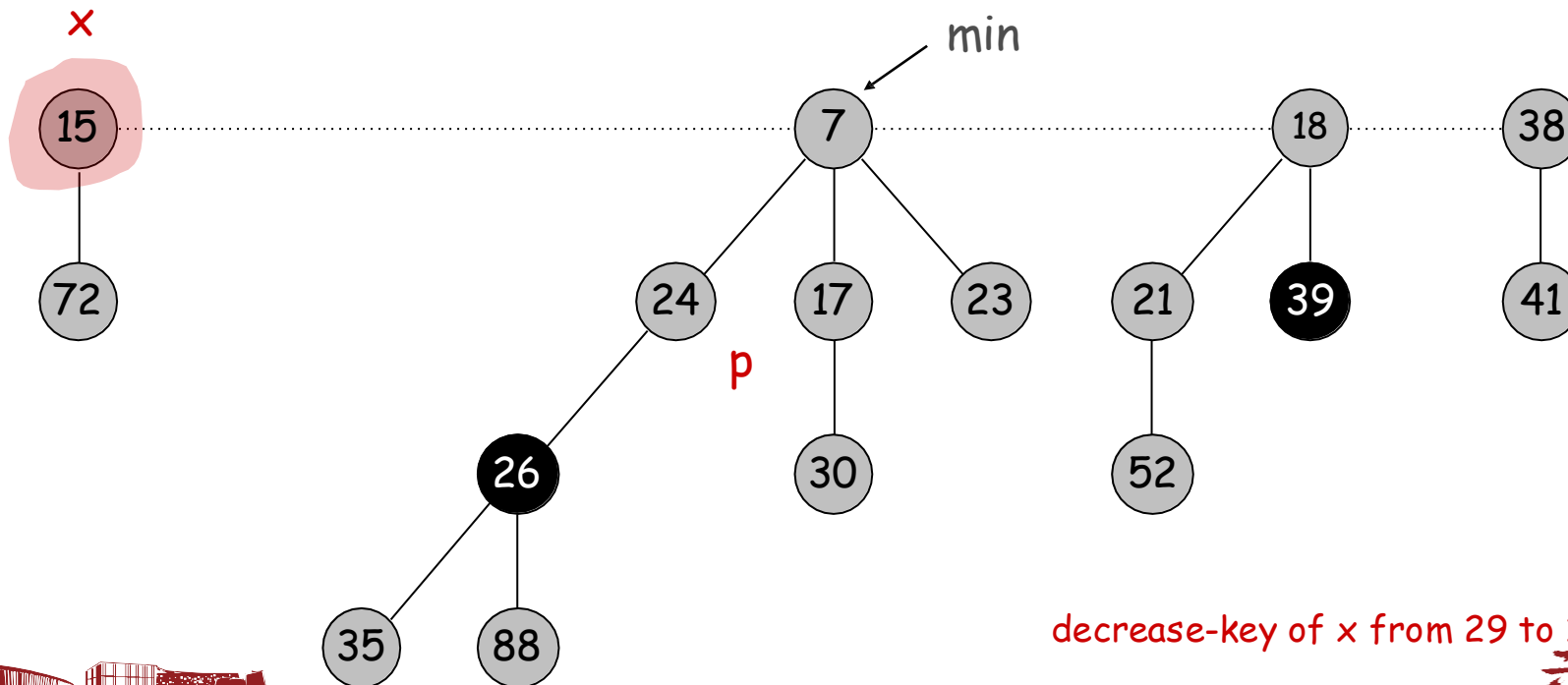


Fibonacci Heaps: Decrease Key



Case 2a. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 29 to 15

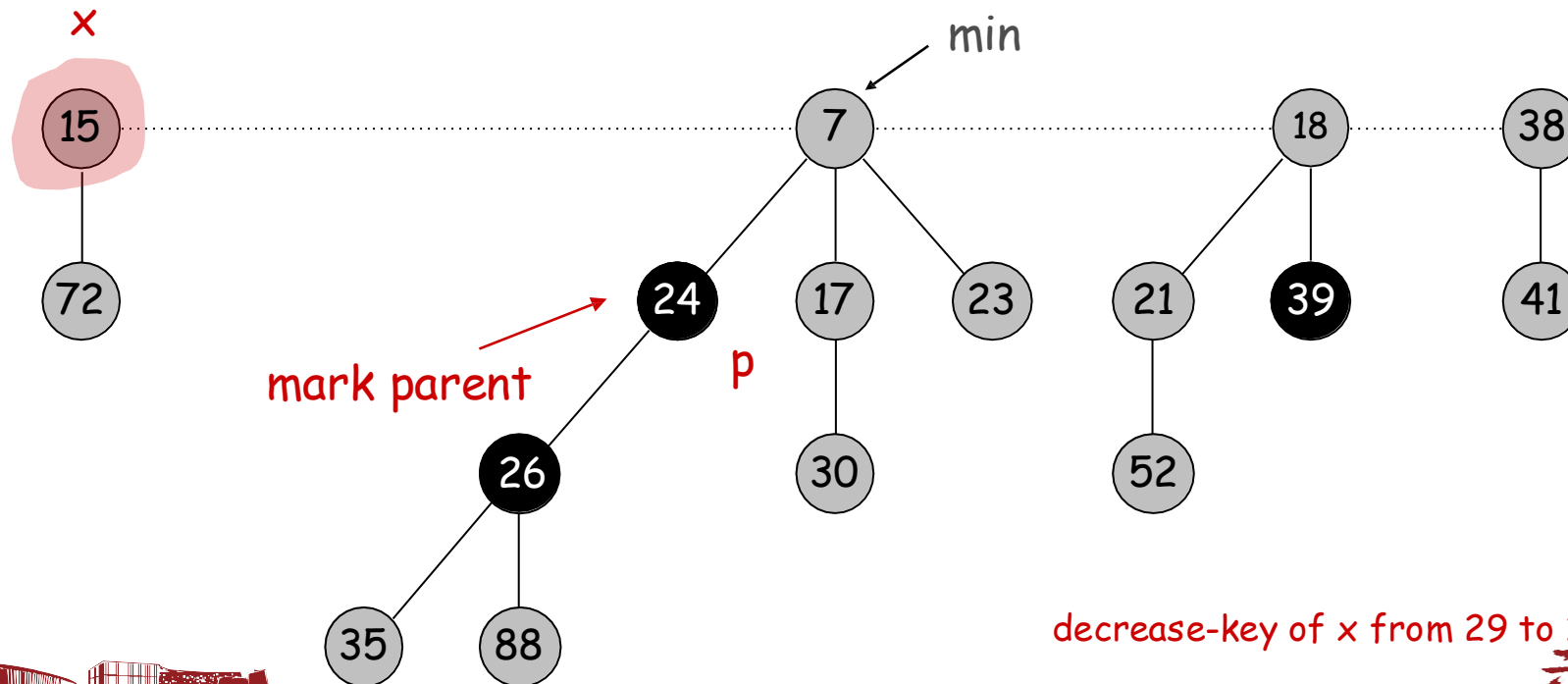


Fibonacci Heaps: Decrease Key



Case 2a. [heap order violated] Decrease

- key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 29 to 15

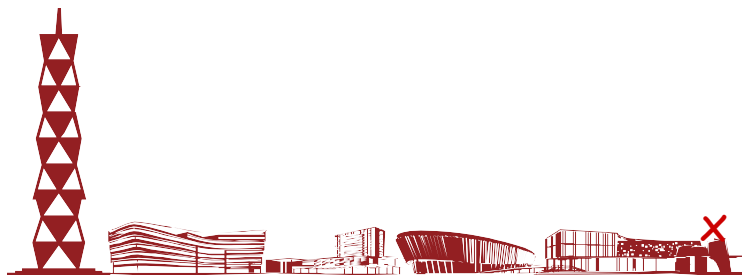
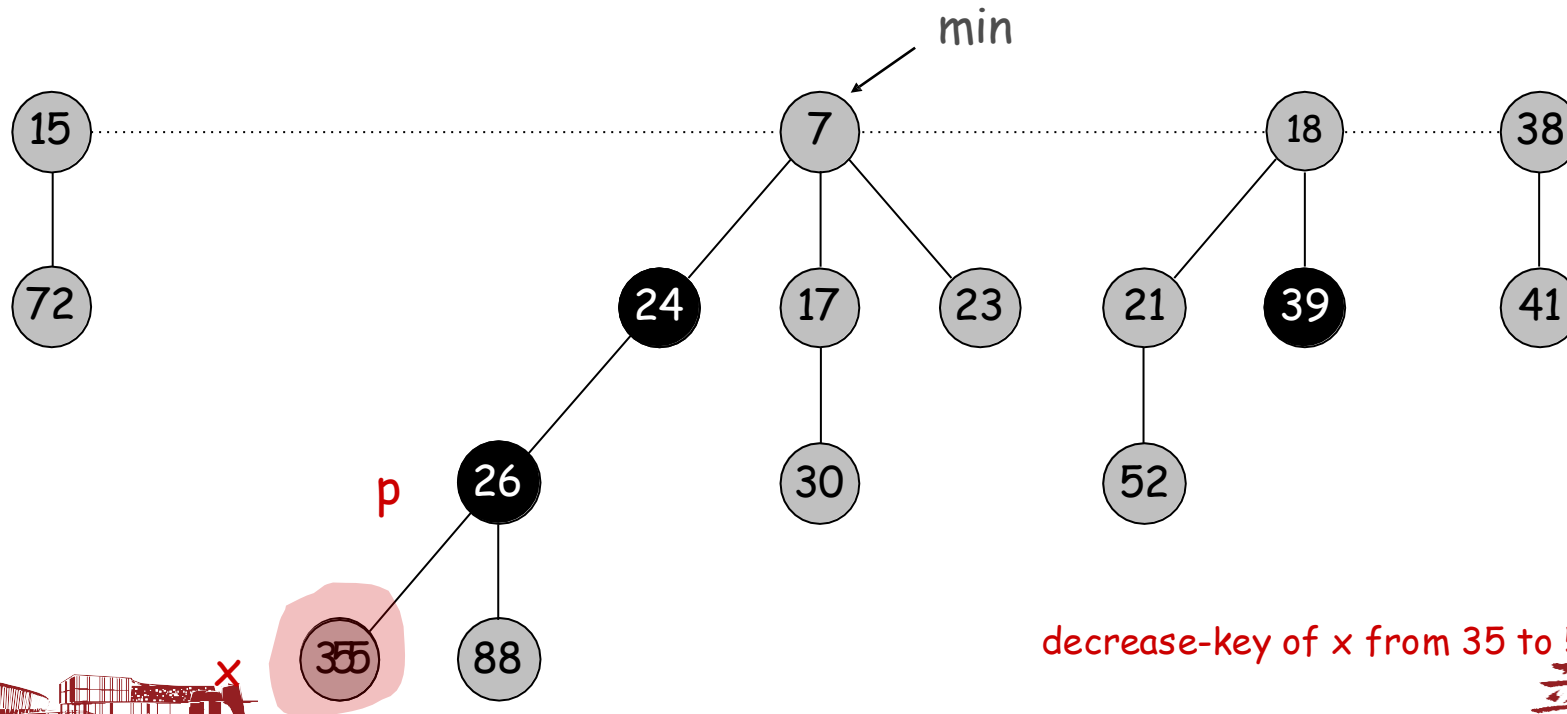


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



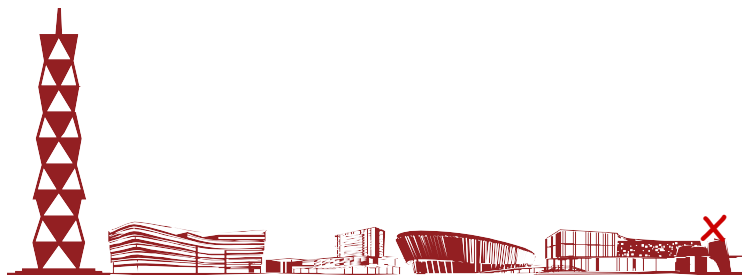
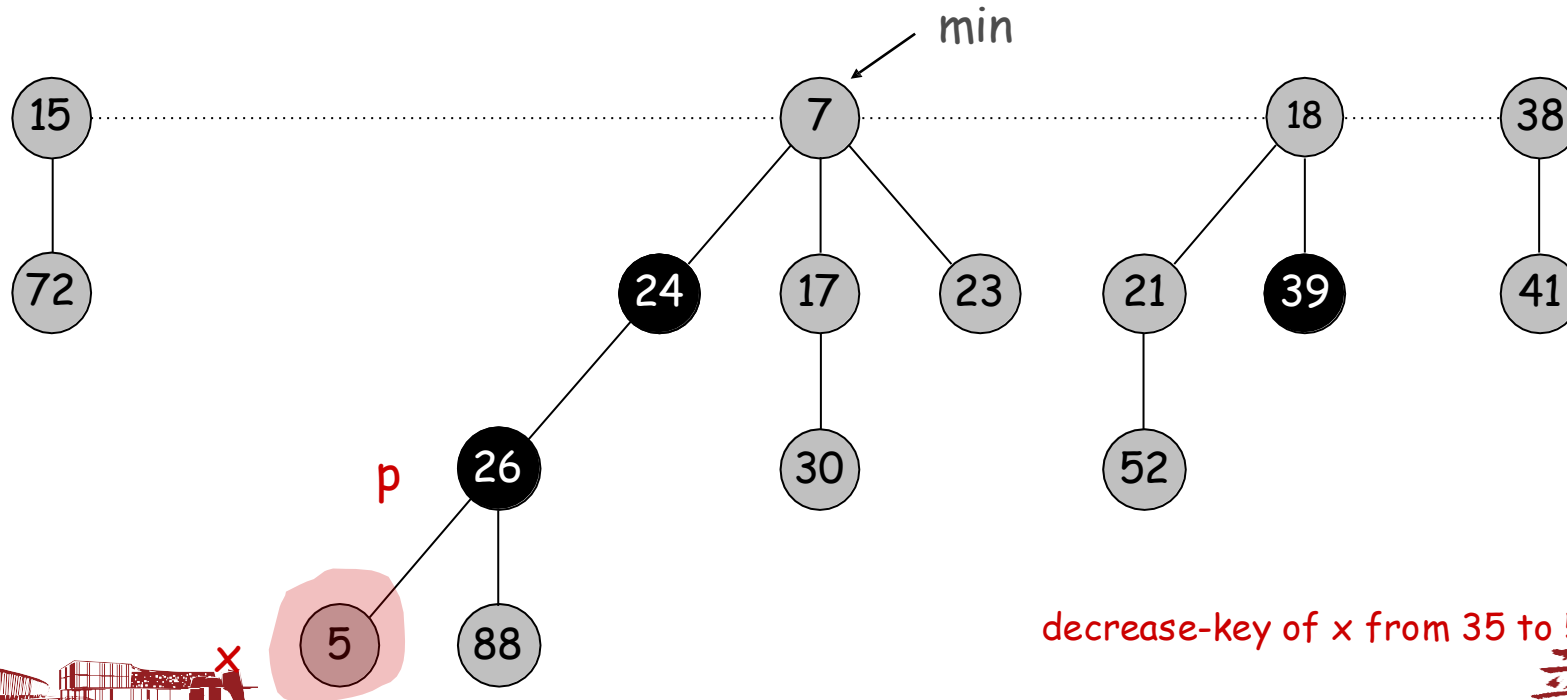


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x.
- Cut tree rooted at x, meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p, meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



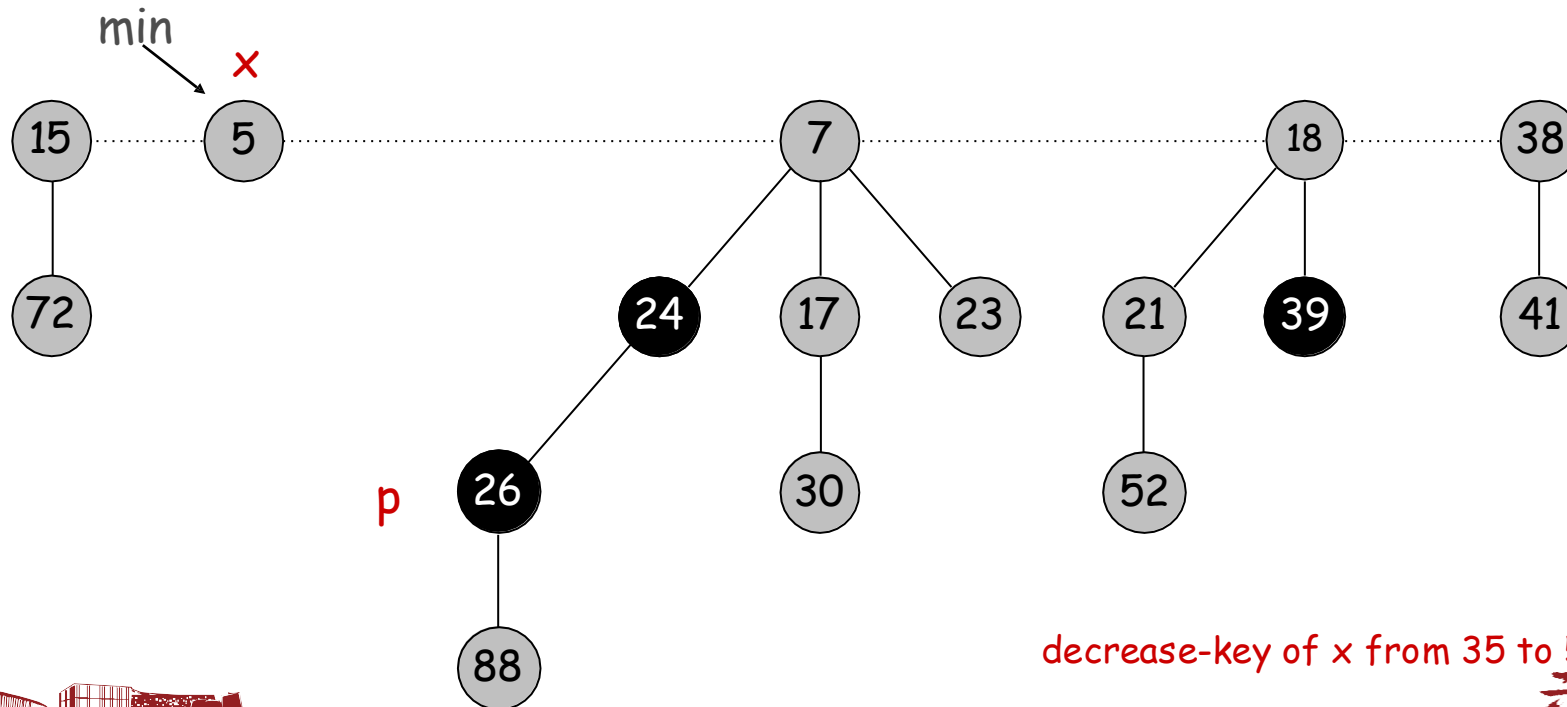


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 35 to 5



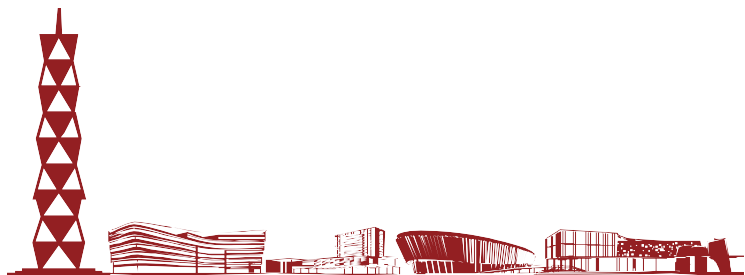
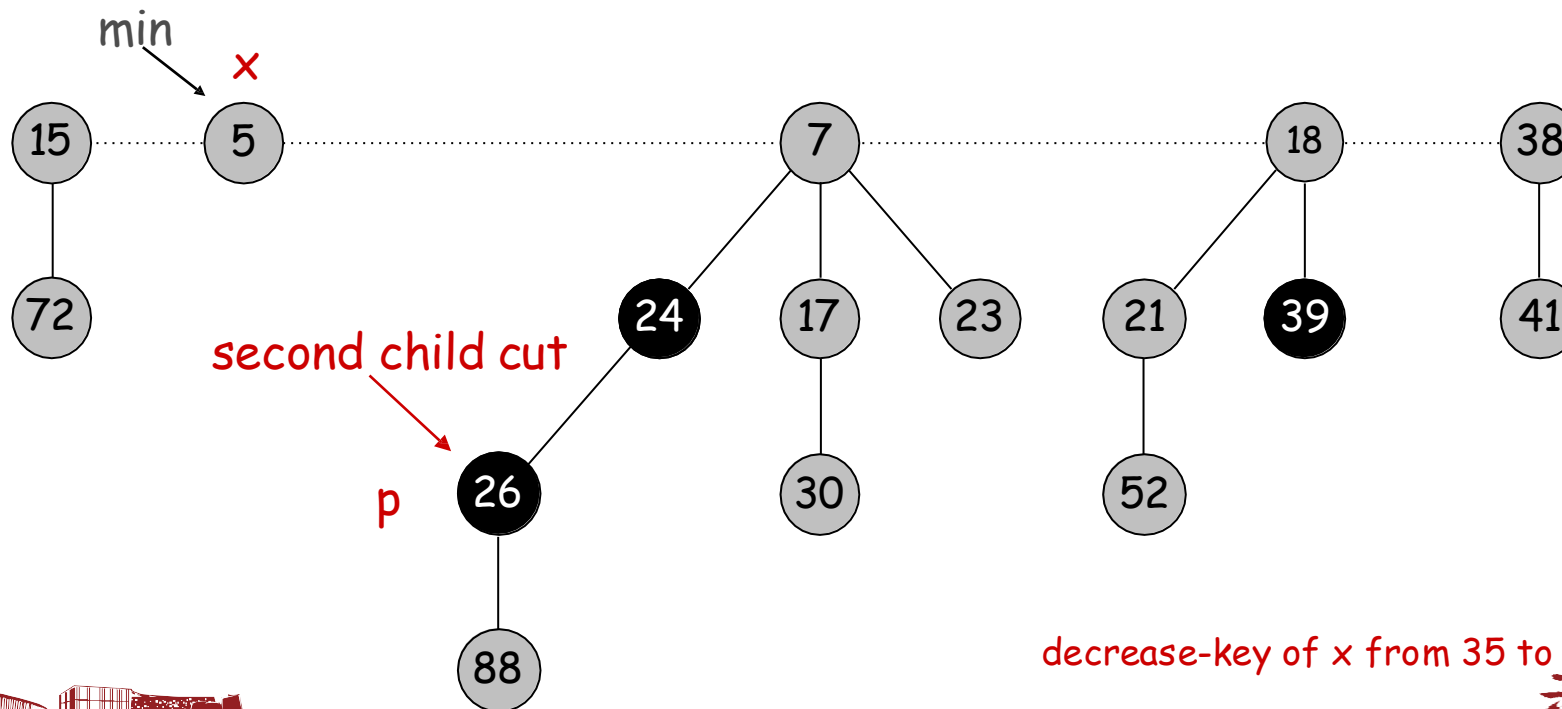


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x.
- Cut tree rooted at x, meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p, meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



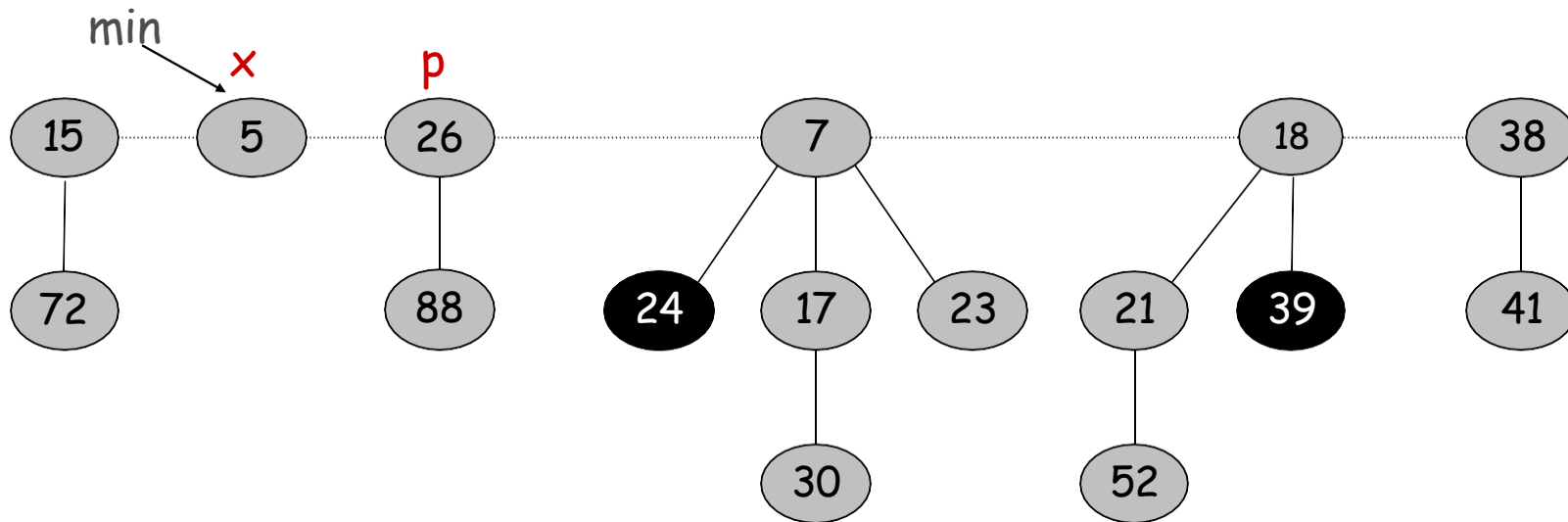


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x.
- Cut tree rooted at x, meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p, meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 35 to 5



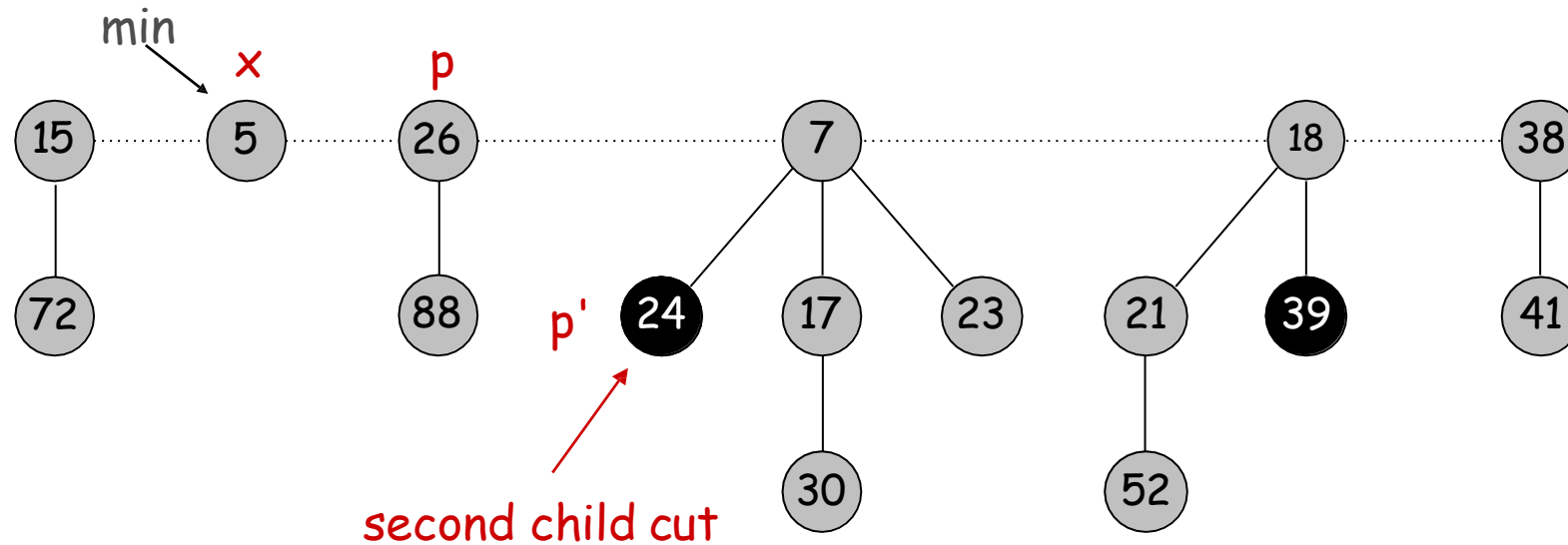


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 35 to 5



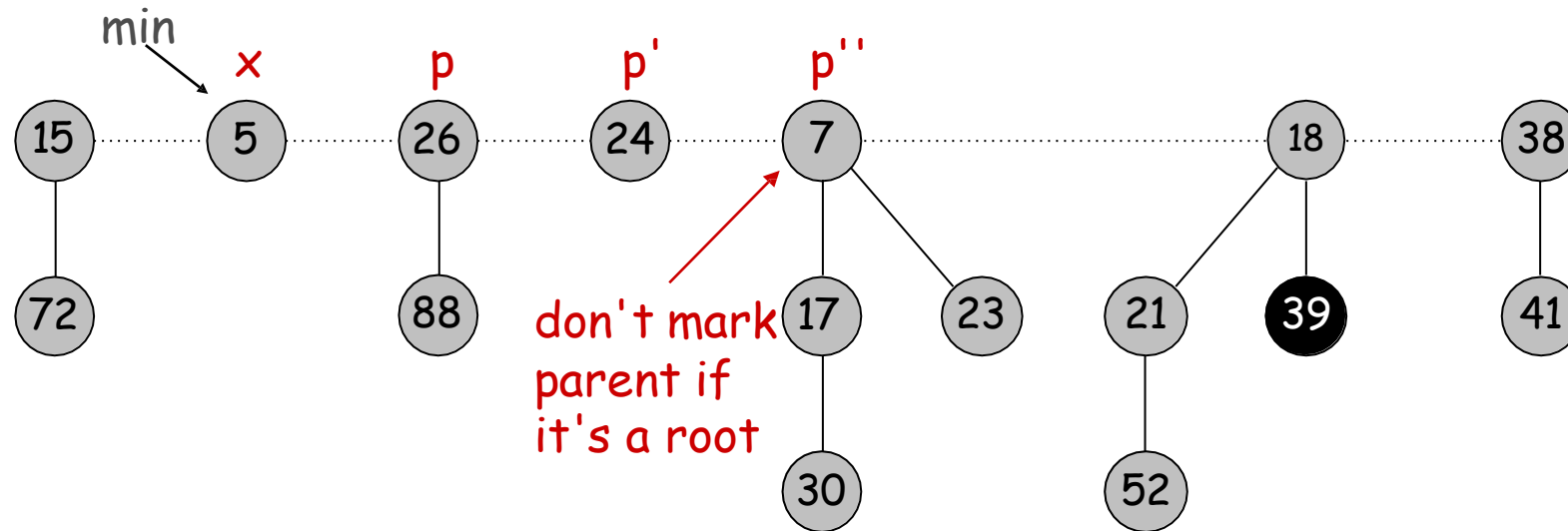


Fibonacci Heaps: Decrease Key



Case 2b. [heap order violated]

- Decrease key of x .
- Cut tree rooted at x , meld into root list, and unmark.
- If parent p of x is unmarked (hasn't yet lost a child), mark it; Otherwise, cut p , meld into root list, and unmark (and do so recursively for all ancestors that lose a second child).



decrease-key of x from 35 to 5





Fibonacci Heaps: Decrease Key Analysis



Let H denote the heap before the decrease key operation.

Cutting out a node takes $O(1)$ time.

Suppose a decrease key operation creates c cascading cuts.

Then the actual cost is $O(c)$.

$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

potential function

For the amortized cost

- Each cut creates one more tree in the root list.
 - It also removes one marked node.
 - After the decrease key, the root list contains $t(H)+c$ trees.
 - It also contains $\leq m(H) - c + 2$ marked nodes.
 - $c-1$ nodes were unmarked by cascading cuts, and the last call to CASCADING-CUT may have marked a node.
 - So the change in potential is $((t(H)) + c + 2(m(H) - c + 2)) - ((t(H) + 2m(H))) = 4 - c$.
 - So the amortized cost is $O(c) + 4 - c = O(1)$
- By scaling the hidden constant in the potential appropriately





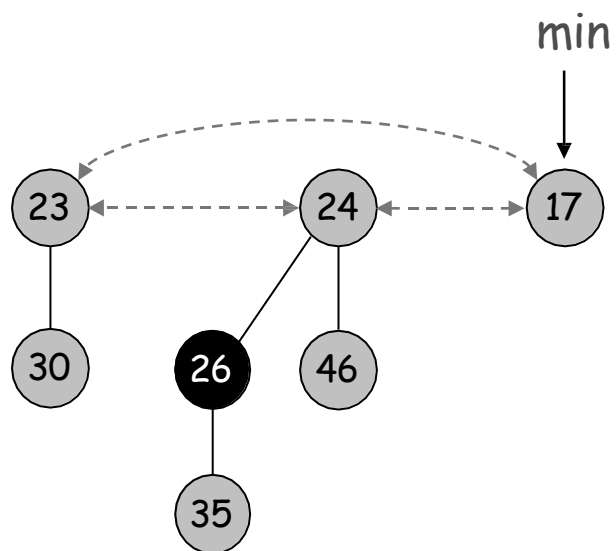
Fibonacci Heaps: Union



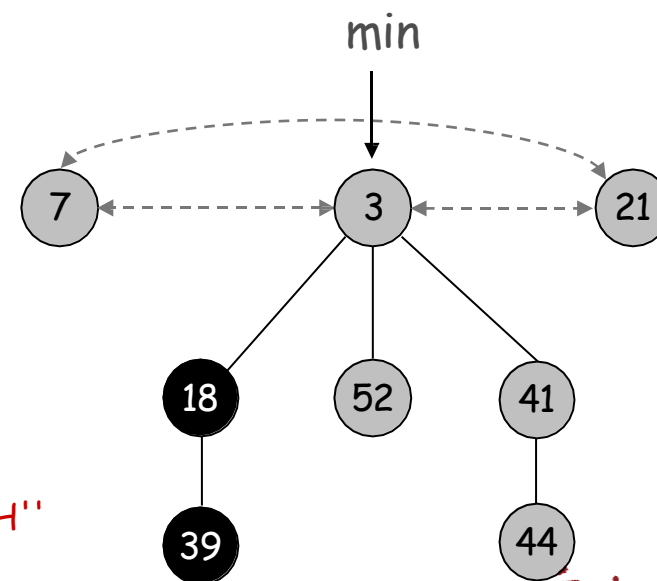
Union. Combine two Fibonacci heaps.

Representation. Root lists are circular, doubly linked lists.

meld



Heap H'



Heap H''





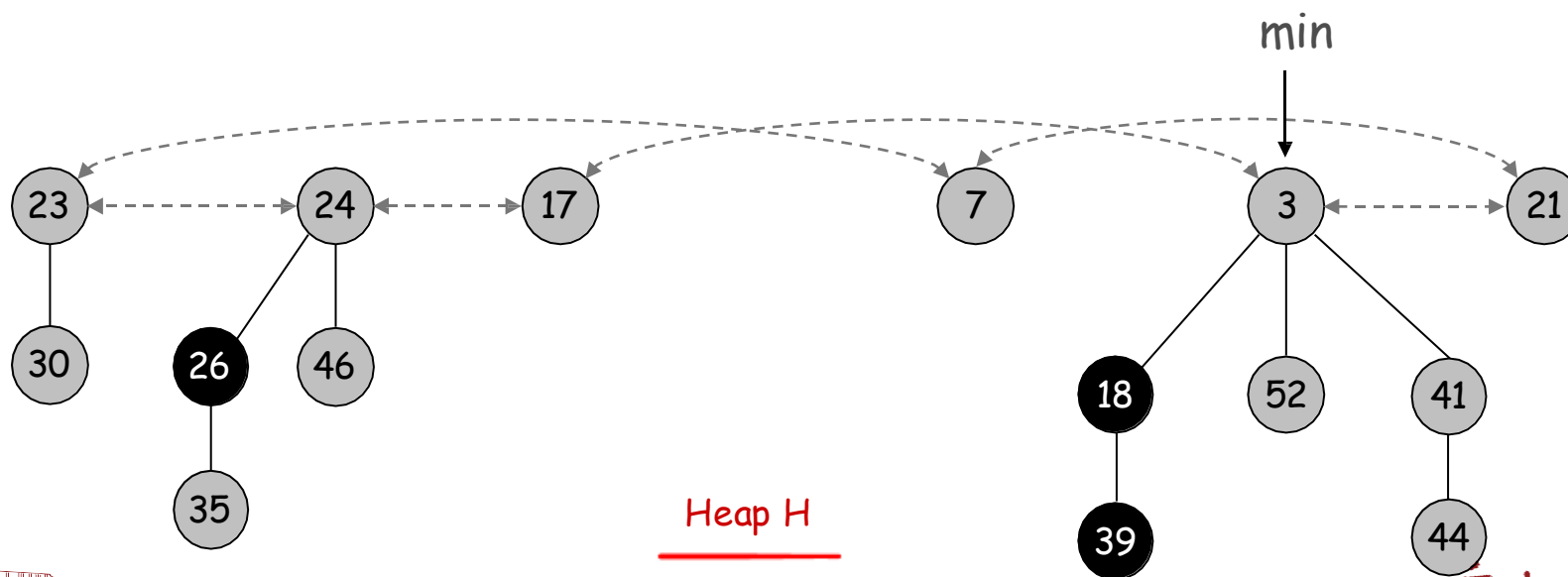
Fibonacci Heaps: Union



Union. Combine two Fibonacci heaps.

Representation. Root lists are circular, doubly linked lists.

0113





Fibonacci Heaps: Union



Union of two heaps

- Concatenate the root lists of the two heaps H_1 and H_2 .
- Set $H.min$ to $\min(H_1.min, H_2.min)$
- Cost = $O(1)$.
- Since new root list is the union of the two old root lists, the change in potential is $\Phi(H') - (\Phi(H_1) + \Phi(H_2)) = t(H') + 2m(H') - (t(H_1) + 2m(H_1) + t(H_2) + 2m(H_2)) = 0$.
- Thus the amortized cost = $O(1)$

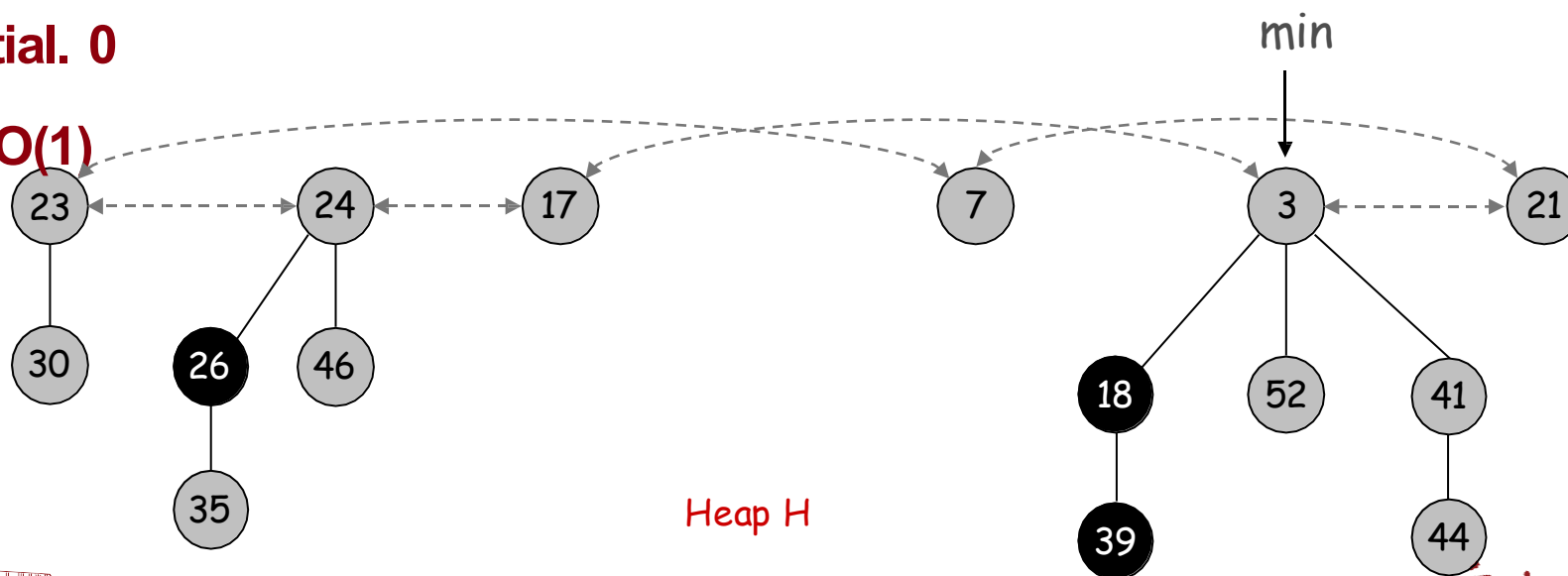
$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

potential function

Actual cost. $O(1)$

Change in potential. 0

Amortized cost. $O(1)$



Heap H



Fibonacci Heaps: Delete



Delete node x .

- decrease-key of x to $-\infty$.
- delete-min element in heap.

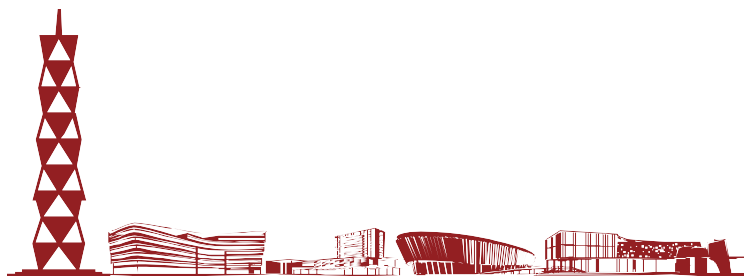
Amortized cost. $O(D(n))$

- $O(1)$ amortized for decrease-key.
- $O(D(n))$ amortized for delete-min.

$$\Phi(H) = \text{trees}(H) + 2 \cdot \text{marks}(H)$$

potential function

$$O(D(n)) = O(\log n)$$



Analysis Summary



Insert. $O(1)$

Delete-min. $O(D(n))$

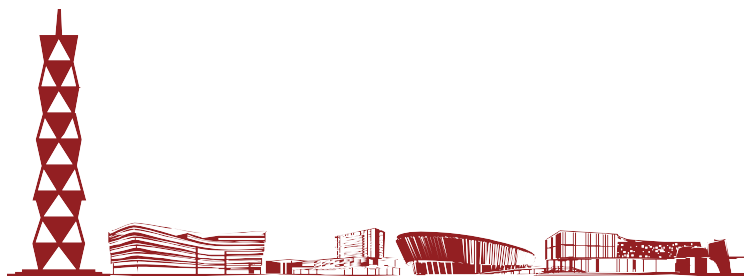
Decrease-key. $O(1)$

Union. $O(1)$

Delete. $O(D(n))$

Key lemma. $D(n) = O(\log n)$.

number of nodes is exponential in degree





Bounding the max degree



- So far, all the operations have $O(1)$ amortized cost, except extract-min (and delete, which calls extract-min).
- Extract-min has amortized cost $O(D(n))$, where $D(n)$ is the max degree of any node in the Fibonacci heap.
- **Def** The golden ratio is $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$.
 - ϕ is the positive solution to the equation $x^2 = x + 1$.
- Recall the Fibonacci F_n is defined by $F_0 = 0, F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.
 - The sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
- **Fact 1** $F_n = \lfloor \frac{\phi^n}{\sqrt{5}} + \frac{1}{2} \rfloor$.
 - For a proof, see section 3.2 of *Introduction to Algorithms*.
- **Fact 2** $F_{n+2} = 1 + \sum_{i=0}^n F_i$.
- **Fact 3** $F_{n+2} \geq \phi^n$.
- We show $D(n) \leq \lfloor \log_\phi n \rfloor$.
- **Def** For any node let $x.\text{deg}$ denote its degree, and $\text{size}(x)$ be the number of nodes in x 's subtree (including x).

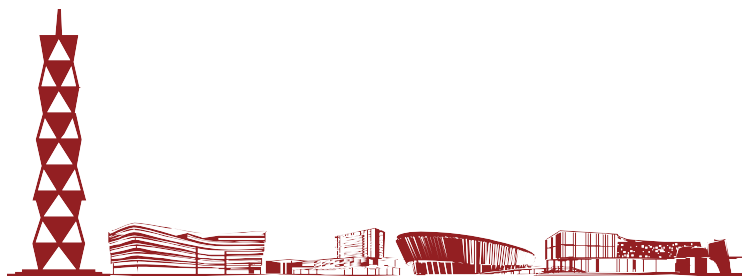




Bounding the max degree



- **Lemma 1** Let x be a node in a Fibonacci heap, and suppose $x.deg = k$. Let $y_1, \dots, y_k$ be the children of x , in the order they were linked to x , from earliest to latest. Then $y_1.deg \geq 0$, and $y_i.deg \geq i - 2$ for $i = 2, \dots, k$.
- **Proof** Obviously $y_1.deg \geq 0$.
 - For $i \geq 2$, when y_i was linked to x , $y_1, \dots, y_{i-1}$ were already children of x , and so x had degree $\geq i - 1$.
 - y_i was linked to x during CONSOLIDATE.
 - So when y_i was linked, we had $y_i.deg = x.deg \geq$ $i - 1$.
 - Since y_i was linked to x , it could have lost at most one child.
 - As soon as y_i loses two children, it's cut and moved to the root list.
 - So $y_i.deg \geq i - 2$.





Bounding the max degree



- **Lemma 2** Let x be a node in a Fibonacci heap and suppose $x.deg = k$. Then $size(x) \geq F_{k+2} \geq \phi^k$.
- **Proof**
 - $s_k = \min \text{size of any node } z \text{ with } degree[z] = k \text{ in a Fibonacci heap, we have } s_0 = 1, s_1 = 2, s_2 = 3, \text{ and } s_k \leq size(z)$
 - Suppose $y_1, y_2, \dots, y_k$ is the child nodes of x and they link to x in order
 - $size(x) \geq s_k = 2 + \sum_{i=2}^k s_{degree[y_i]} \geq 2 + \sum_{i=2}^k s_{i-2}$ because of Lemma 1: $degree[y_i] \geq i - 2$ and $s_{degree[y_i]} \geq s_{i-2}$
 - By induction to prove for any $k, s_k \geq F_{k+2}$
 - Base: $k=0$ and $k=1$, it holds
 - By induction, suppose $k \geq 2$ and $s_i \geq F_{i+2}, i = 0, 1, \dots, k-1$, we have
 - $s_k \geq 2 + \sigma_{i=2}^k s_{i-2} \geq 2 + \sigma_{i=2}^k F_i = 1 + \sigma_{i=0}^k F_i = F_{k+2} \geq \phi^k$
 - The last two equalities follow by Facts 2 and 3.
- For any n node Fibonacci heap H , the max degree $D(n) = O(\log n)$.
- **Proof** Let x be any node in H , and let $k = x.deg$.
 - We have $n \geq size(x) \geq \phi^k$.
 - So $k \leq \lfloor \log_{\phi} n \rfloor$.

